Laporan Tugas Besar 1

IF3070 Dasar Inteligensi Artifisial 2026/2027



Disusun oleh:

Ghaisan Nufail	(18224090)
Andreas Saputra Tambun	(18224110)
Daniel Wicaksana Godjali	(18224114)

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA - KOMPUTASI

INSTITUT TEKNOLOGI BANDUNG

2026

Deskripsi Persoalan



Gambar 1. 1 (Sumber: Gambar Kapal)

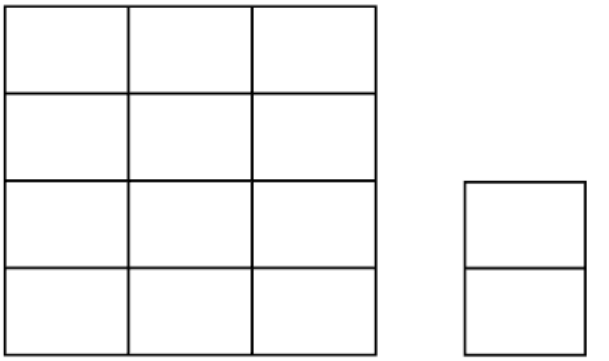
Anda adalah seorang *Forward Deployed Engineer* (FDE) di PT ~~Owen~~ Intim Line yang mengoperasikan kapal kargo pengangkut kendaraan. Perusahaan baru saja memperoleh kontrak pengiriman dari beberapa dealer kendaraan sekaligus. Pada setiap jadwal keberangkatan, perusahaan menerima daftar kendaraan dengan dimensi, berat, tarif pengiriman, dan deadline yang berbeda-beda.

Selama ini, penempatan kendaraan pada geladak kapal dilakukan secara manual. Akibatnya, ruang geladak sering tidak termanfaatkan dengan baik dan keputusan pemilihan letak kendaraan belum tentu menghasilkan keuntungan yang optimal.

Tim operasional meminta Anda membangun sistem pendukung keputusan untuk menentukan kendaraan yang akan diangkut serta posisi dan orientasi setiap kendaraan pada geladak. Sistem tersebut harus dapat mengevaluasi berbagai kemungkinan konfigurasi, mempertimbangkan fungsi-fungsi objektif yang ditentukan, dan menghasilkan solusi penataan yang optimal.

Pembahasan

Setiap kendaraan dan geladak direpresentasikan sebagai grid dua dimensi berbentuk persegi atau persegi panjang. Solusi dapat memilih kendaraan mana yang diangkut dan menentukan posisi serta orientasinya di dalam geladak, dengan tetap memenuhi seluruh batasan.



Kapal Kendaraan

Daftar Variabel Permasalahan

Entitas	Atribut	Contoh	Keterangan
vehicle	id	"V1"	*Identifier*
	width	2	Ukuran lebar dari kendaraan (satuan grid, secara default ukuran horizontal)
	length	3	Ukuran panjang dari kendaraan (satuan grid, secara default ukuran vertikal)
	shipping_fee	100	Tarif/keuntungan pengiriman kendaraan (satuan juta rupiah)

	weight	20	Berat kendaraan (satuan unit)
	eta	5	Estimated Time of Arrival, perkiraan waktu kedatangan kendaraan (dalam sisa hari)
placement	loaded	true	Menentukan apakah sebuah kendaraan dimasukan ke geladak kapal pada sebuah state (false berarti x dan y harus None)
	x	2	Letak koordinat x (horizontal) dari sebuah kendaraan (setiap koordinat berpatokan di kiri atas objek)
	y	4	Letak koordinat y (vertikal) dari sebuah kendaraan (setiap koordinat berpatokan di kiri atas objek)
	rotated	false	Variabel penentu perlakuan *width* dan *length* dari kendaraan (true berarti width dan length ditukar)
ship	width	5	Ukuran lebar/horizontal dari geladak kapal (satuan gridl)
	length	10	Ukuran panjang/vertikal dari geladak kapal (satuan grid)
	max_capacity	50	Kapasitas angkut maksimum suatu kapal

			(satuan unit)
state	placements	{"V1": placement}	Dictionary/Map yang berisi daftar *placement* dari setiap kendaraan

Objective Function & Constraints

- **Objective function**

  Permasalahan ini bertujuan untuk memaksimalkan *ShippingFee* pada kapal.

- **Kendaraan tidak boleh saling bertumpuk (*no overlapping*)**

  Tidak boleh ada satu sel grid geladak yang ditempati oleh lebih dari satu kendaraan.

- **Kendaraan harus berada di dalam batas geladak.**

  Seluruh sel grid yang ditempati suatu kendaraan harus berada di dalam dimensi geladak kapal.

- **Kapal tidak boleh mengangkut melebihi kapasitasnya.**

Spesifikasi *Ship*:

- width : 5
- length : 15
- max_capacity : 100

Daftar *Vehicles*:

id	width	length	shipping_fee	weight	eta
V1	2	3	200	15	5
V2	3	7	350	25	2

V3	1	1	50	10	1
V4	2	4	200	15	3
V5	10	3	500	40	5
V6	8	4	400	35	5
V7	1	5	150	15	2
V8	5	5	250	20	1
V9	1	6	150	15	3
V10	7	2	300	25	1

Dokumentasi eksperimen bisa diakses melalui repo github di folder output/

Hill Climbing

Penjelasan Implementasi

Algoritma Steepest Ascent Hill-climbing ini diimplementasikan dengan fungsi *hill_climbing(vehicles, ship, initial_state=None)*. Fungsi tersebut menerima data dari kendaraan, spesifikasi kapal, dan juga state awal opsional. Jika state awal tidak diberikan, maka konfigurasi akan dibentuk secara acak dengan *random_state()*. Konfigurasi tersebut diperiksa dengan menggunakan *is_valid()* untuk memastikan batas dari geladak, kapasitas angkut, dan ketentuan tidak bertumpuk-nya terpenuhi.

Di setiap iterasi, algoritma mengevaluasi seluruh tetangga valid yang dihasilkan *generate_neighbors()*, kemudia akan memilih tetangga dengan total *shipping fee* tertinggi. Perpindahan diterima hanya jikalau nilainya lebih tinggi daripada state saat ini. Pencarian akan berhenti jika tidak terdapat tetangga valid atau tidak ditemukan adanya peningkatan nilai objektif.

Fungsi hill_climbing()

```
def hill_climbing(vehicles, ship, initial_state=None):
    """Steepest ascent: tetangga dengan nilai lebih tinggi. Pencatatan setiap putaran
pemeriksaan."""
    started = perf_counter()

    if initial_state is None:
        current = random_state(vehicles, ship)
    else:
        current = initial_state.copy()

    if not is_valid(current, vehicles, ship):
        raise ValueError("State awal tidak valid.")

    start_state = current.copy()
    current_score = objective(current, vehicles)
```

```
    history = [current_score]
    iterations = 0
    accepted_moves = 0

    while True:
        iterations += 1
        neighbors = generate_neighbors(current, vehicles, ship)

        if not neighbors:
            history.append(current_score)
            stop_reason = "Tidak ada tetangga valid."
            break

        best = max(neighbors, key=lambda state: objective(state, vehicles))
        best_score = objective(best, vehicles)

        if best_score <= current_score:
            history.append(current_score)
            stop_reason = "Tidak ada tetangga dengan nilai lebih tinggi."
            break

        current = best
        current_score = best_score
        accepted_moves += 1
        history.append(current_score)

    return {
        "initial_state": start_state,
        "final_state": current.copy(),
        "initial_score": history[0],
        "final_score": current_score,
        "history": history,
        "iterations": iterations,
        "accepted_moves": accepted_moves,
        "duration_seconds": perf_counter() - started,
        "stop_reason": stop_reason,
    }
```

Fungsi mengembalikan *dictionary* yang memuat state awal dan akhir, nilai objektif, riwayat pencarian, jumlah iterasi, jumlah perpindahan, durasi, dan alasan pengentian. Variabel *iterations* berguna untuk menghitung setiap putaran pemeriksaan, termasuk pemeriksaan terakhir yang menghentikan pencarian, sedangkan kalau *accepted_moves* hanya menghitung perpindahan yang diterima. Riwayat dalam *history* mencatat nilai

awal dan dan nilai setiap pemeriksaan. Oleh karena itu, nilai terakhir bisa berulang ketika pencarian berhenti tanpa perpindahan.

Durasi diukur dengan *perf_counter()*, dari pembentukan atau penyalinan state awal sampai pencarian selesai. Pengukuran termasuk validasi dan evaluasi tetangga, sedangan pembuatan visualisasi serta penyimpanan berkas dilakukan setelahnya dengan *HC_out()* dan tidak termasuk dalam durasi pencarian.

Pemanggilan dan Dokumentasi Hasil

Fungsi *HC_out(vehicles, ship, seed=None)* berguna untuk mengatur eksekusi dan dokumentasi hasil run algoritma. Jika seed diberikan, fungsi akan memanggil *random.seed(seed)* sebelum menjalankan *hill_climbing()*. Ini memungkinkan konfigurasi awal dihasilkan kembali selama kode, data, dan lingkungan dari eksekusinya sama. Fungsi tersebut dipanggil dengan *run_hc()*.

```
def HC_out(vehicles, ship, seed=None):
    if seed is not None:
        random.seed(seed)

    result = hill_climbing(vehicles, ship)
```

Hasil pencarian akan ditampilkan sebagai ringkasan terminal, gambar state awal dan akhir dengan *plot_state()*, dan grafik nilai objektif berdasarkan *history*. Fungsi *_create_experiment_dir()* membuat folder *output/hc/eksperimen_x* dengan x adalah satu angka lebih besar dari nomor eksperimen tertinggi yang sudah ada, dan penomoran dimulai tentunya dari 1 jika belum terdapat folder eksperimen. Setiap folder menyimpan ketiga gambar hasil run dan satu berkas *hasil.json*. Berkas JSON berisi konfigutasi state dan data numerik untuk analisis.

```
    next_number = max(experiment_numbers, default=0) + 1
    output_dir = output_root / f"eksperimen_{next_number}"
    output_dir.mkdir()

    return output_dir
```

Pengaturan dan Rekap Hasil Eksperimen

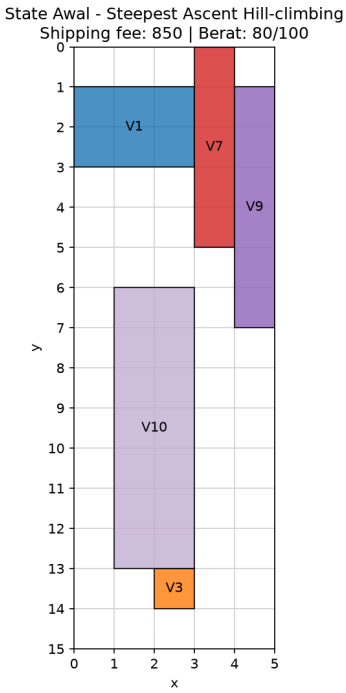
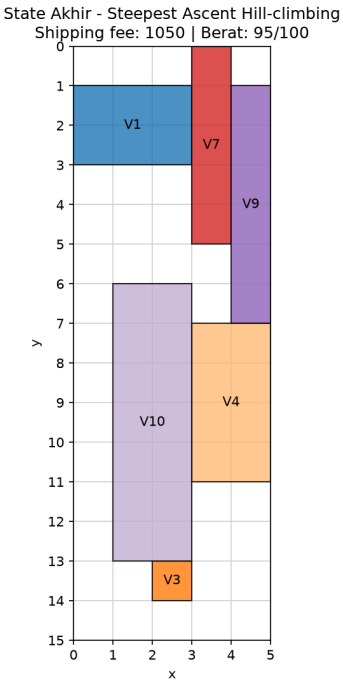
Pengujian atau eksperiman dilakukan sebanyak tiga kali dengan seed 1, 2, dan 3. Semua percobaan menggunakan sepuluh kendaraan yang sama, dimensi geladak 5x15 satuan grid, dan kapasitas maksimum yaitu 100 satuan berat. Perbedaan seed digunakan untuk melihat hasil pencarian dari konfigurasi awal yang berbeda.

Tabel berikut menyajikan hasil eksperimen. Nilai objektif adalah total *shipping fee* kendaaran dimuat, dalam satuan juta rupiah.

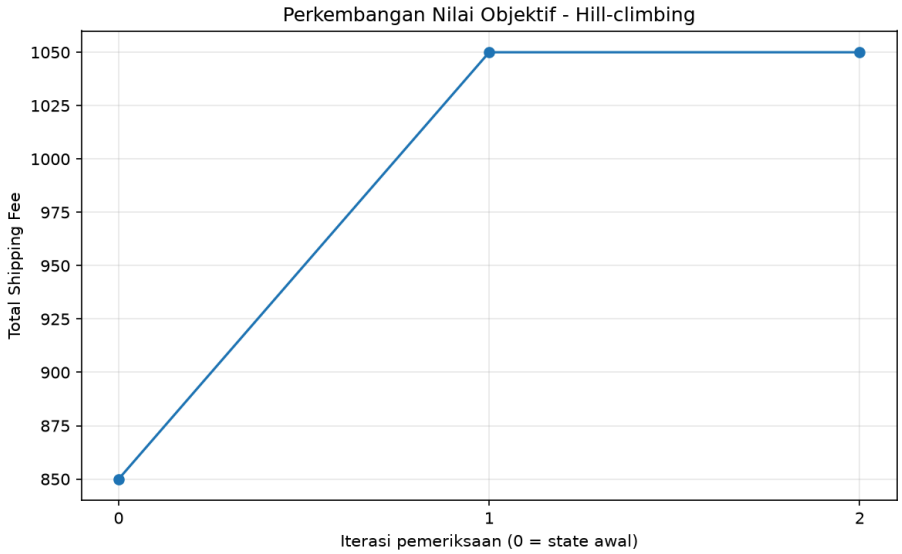
Eksperimen	Nilai Awal	Nilai Akhir	Iterasi pemeriksaan	Perpindahan diterima	Durasi (detik)
1 (seed 1)	850	1050	2	1	0,010065
2 (seed 2)	1050	1050	1	0	0,004933
3 (seed 3)	1050	1100	2	1	0,009615

Ketiga percobaan itu berhenti karena tidak ditemukan adanya tetangga dengan nilai objektif yang lebih tinggi daripada state terakhir.

Eksperimen 1

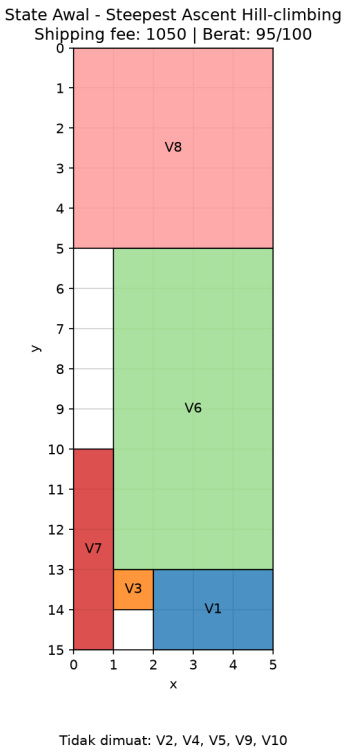
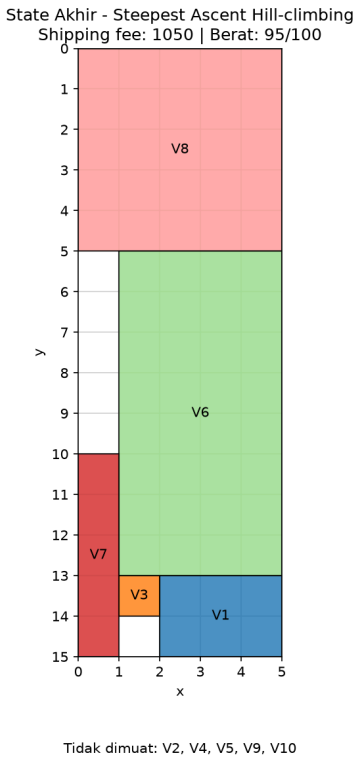
State Awal	State Akhir
 Tidak dimuat: V2, V4, V5, V6, V8	 Tidak dimuat: V2, V5, V6, V8

Perkembangan Nilai Objektif:

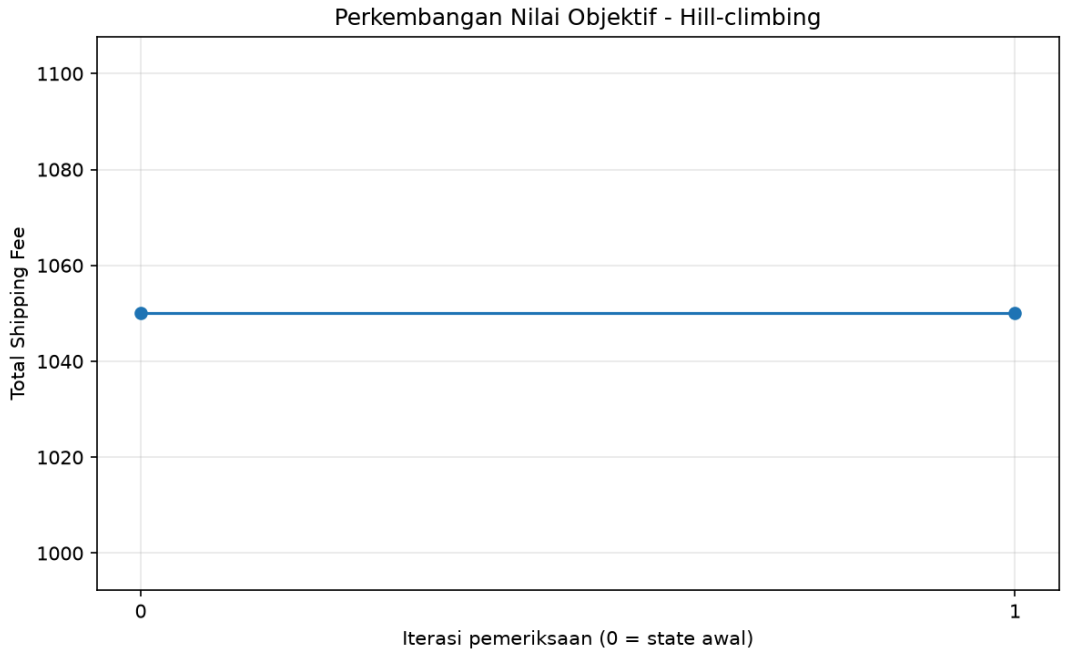


Pada seed 1, algoritma meningkatkan nilai objektif dari 850 menjadi 1050 dengan tambahan kendaraan V4. Total berat naik dari 80 menjadi 95, masih dalam kapasitas maksimum yaitu 100. Riwayat [850, 1050, 1050] menunjukkan satu perpindahan pada pemeriksaan pertama, lalu pencarian berhenti pada pemeriksaan kedua karena tidak menemukan peningkatan berikutnya.

Eksperimen 2

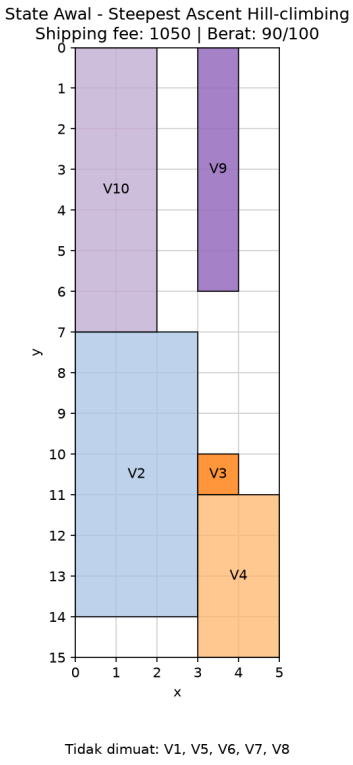
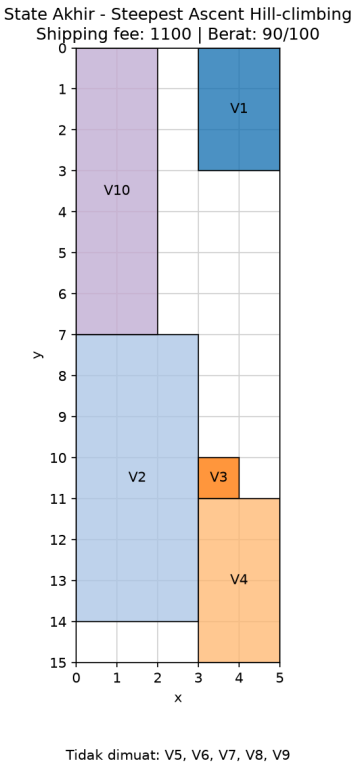
State Awal	State Akhir
	

Perkembangan Nilai Objektif:



Pada seed 2, konfigurasi awal bernilai 1050 tidak memiliki tetangga dengan nilai objektif yang lebih tinggi. Algoritma berhenti setelah adanya satu pemeriksaan tanpa menerima perpindahan, sehingga state awal dan akhir jadinya tetap sama. Grafik [1050, 1050] mencatat pemeriksaan tersebut meskipun tidak terjadi perubahan state.

Eksperimen 3

State Awal	State Akhir
	

Perkembangan Nilai Objektif:



Pada seed 3, nilai objektif meningkat dari 1050 menjadi 1100 dengan satu perpindahan. Riwayat [1050, 1100, 1100] menunjukkan adanya peningkatan pada pemeriksaan pertama dan penghentian pada pemeriksaan kedua. Nilai akhir 1100 merupakan hasil tertinggi dari ketiga percobaan Hill-Climbing.

Analisis Eksperimen Hill-climbing

Ketiga eksperimen tersebut menghasilkan nilai objektif akhir antara 1050 dan 1100, dengan rata-rata 1066,67. Peran konfigurasi awal terlihat pada eksperimen 2 dan 3, keduanya dimulai dengan nilai 1050, tetapi hanya eksperimen 3 yang memperoleh peningkatan. Hal ini menunjukkan bahwa kesamaan nilai objektif awal tidak menjamin kesamaan hasil pencarian. Susunan kendaraan menentukan ruang yang tersedia dan tetangga valid yang dapat meningkatkan nilai objektif.

Kemampuan eksplorasi dibatasi oleh aturan penerimaan perpindahan. Perubahan posisi atau orientasi tanpa mengubah kendaraan yang dimuat menghasilkan nilai objektif yang sama, sehingga tidak diterima. Padahal, penataan ulang tersebut mungkin diperlukan sebelum kendaraan lain dapat dimasukkan. Karena itu, pencarian dapat berhenti meskipun konfigurasi yang lebih baik masih mungkin tersedia melalui beberapa langkah. Nilai akhir tertinggi sebesar 1100 belum membuktikan tercapainya optimum global.

Pencarian berlangsung selama 0,004933–0,010065 detik, dengan rata-rata sekitar 0,008204 detik. Seluruh percobaan berhenti setelah satu atau dua iterasi pemeriksaan. Durasi tersebut mencakup pembentukan state awal dan evaluasi tetangga, sehingga jumlah iterasi saja tidak menjelaskan seluruh perbedaan waktu eksekusi. Karena setiap seed hanya dijalankan sekali, pengukuran ini menggambarkan

durasi ketiga percobaan dan belum cukup untuk menyimpulkan pola waktu eksekusi secara umum.

Simulated Annealing

Pendekatan *Simulated Annealing* (SA) diimplementasikan pada fungsi 'simulated_annealing()' yang berada di file 'src/search/SA.py'. Berbeda dengan *Hill Climbing* yang hanya menerima neighbor dengan nilai yang lebih tinggi, SA juga dapat menerima neighbor yang lebih buruk dengan peluang e^(ΔE/T), dengan ΔE adalah selisih nilai neighbor terhadap state saat ini dan T adalah temperatur. Temperatur diturunkan secara bertahap, pada temperatur tinggi algoritma masih mudah berpindah ke state yang lebih buruk sehingga berpeluang keluar dari local optima, sedangkan pada temperatur rendah perilakunya mendekati *Hill Climbing*.

Pendekatan melalui SA dilakukan dengan ketentuan berikut:

- Temperatur awal (T0) ditetapkan 800, faktor pendinginan geometrik (alpha) 0,998, dan temperatur minimum (T_min) 1,0
- Penurunan temperatur menggunakan geometrik T dengan cara menghitung alpha × T pada setiap iterasi
- Eksperimen dijalankan sebanyak 3 kali menggunakan seed 1, 2, dan 3.
- Setiap iterasi memilih satu tetangga valid secara acak dari seluruh tetangga yang dibangkitkan oleh `generate_neighbors()`
- Rekap hasil setiap eksperimen disimpan dalam bentuk JSON beserta gambar *state* dan grafik

Tabel Parameter Eksperimen Simulated Annealing

Parameter	Nilai	Keterangan
T0	800	Temperatur awal
alpha	0,998	Faktor pendingin geometrik

T_min	1,0	Temperatur minimum; batas search jika T < T_min
max_iter	5000	Batas iterasi maksimum
seed	1, 2, 3	seed yang digunakan tiap eksperimen/run

Penjelasan Implementasi

Algoritma SA mencari konfigurasi dengan *total shipping fee* maksimum melalui state yang terus diperbarui, satu per satu. Pada tiap iterasi satu neighbor dipilih secara acak, kemudian ditentukan apakah state saat ini dapat pindah ke neighbor tersebut berdasarkan kriteria SA, neighbor yang lebih baik (ΔE > 0) selalu diterima, sedangkan neighbor dengan ΔE ≤ 0 diterima dengan peluang e^(ΔE/T). Karena e^0 = 1, tetangga dengan nilai objektif sama (*sideways move*) selalu diterima.

Fungsi 'simulated_annealing(vehicles, ship, T0, alpha, T_min, max_iter, seed)' memakai fungsi bersama pada 'src/utils.py', yaitu 'random_state()' untuk *initial state*, 'objective()' untuk menghitung total *shipping fee* kendaraan yang dimuat, serta 'random_neighbor()' untuk memilih neighbor. Fungsi 'random_neighbor()' memanggil 'generate_neighbors()' yang menghasilkan seluruh neighbor valid dari tiga jenis perubahan, yaitu memindahkan kendaraan ke koordinat lain (termasuk memasukkan kendaraan yang belum dimuat) atau mengeluarkannya dari kapal, mengubah orientasi kendaraan, dan menukar penempatan dua kendaraan baik di dalam maupun di luar kapal. Setiap kandidat diperiksa 'is_valid()' sehingga tetangga yang dipilih selalu memenuhi constraint menumpuk, batas geladak, dan kapasitas kapal, dan SA tidak perlu menolak neighbor karena melanggar batasan. Pemilihan dilakukan seragam di antara tetangga valid, sehingga jenis perubahan yang jumlahnya paling banyak, paling sering terpilih.

Inisialisasi dan Validasi Parameter

```
# T geometrik: T <- alpha * T. berhenti kalo T < T_min (anggap T ~ 0) atau
```

```
iterasi selesai
def simulated_annealing(vehicles, ship, T0, alpha, T_min, max_iter, seed):
    if T0 <= 0 or T_min <= 0 or not 0 < alpha <= 1:
        raise ValueError("T0, T_min harus > 0 dan 0 < alpha <= 1")

    random.seed(seed)
    start_time = perf_counter()

    current = random_state(vehicles, ship)
    current_value = objective(current, vehicles)
    initial_state = current.copy()

    value_history = [current_value]
    temperature_history = []
    probability_history = []  # e^(dE/T), None kalo neighbor lebih baik
    worse_considered = 0
    worse_accepted = 0
    stuck_count = 0  # hitung neighbor lebih buruk yang ditolak
    iterations = 0
    T = T0
```

Fungsi memeriksa parameter, T0 dan T_min harus positif dan alpha harus berada pada rentang 0 < alpha ≤= 1. Pelanggaran menghasilkan 'ValueError' agar pendinginan tidak menjadi tak terhingga atau menaikkan temperatur.

Pemanggilan 'random.seed(seed)' berada di dalam fungsi, sehingga setiap run dapat diulang secara persis selama kode, data, dan parameter sama. *Initial state* dibentuk oleh 'random_state()' dan disalin ke 'initial_state' untuk dokumentasi. Fungsi menyiapkan 'value_history' (nilai state saat ini pada tiap iterasi, dengan indeks nol sebagai *initial state*), 'temperature_history', dan 'probability_history' untuk menyimpan e^(ΔE/T) pada langkah yang tidak lebih baik. Penghitung 'worse_considered', 'worse_accepted', dan 'stuck_count' mencatat jumlah neighbor lebih buruk yang diperiksa, diterima, dan yang ditolak.

Pemilihan Neighbor dan Kriteria

```
for _ in range(max_iter):
        if T < T_min:
            break
        iterations += 1
        temperature_history.append(T)
        probability_history.append(None)

        neighbor = random_neighbor(current, vehicles, ship)
        if neighbor is not None:  # None -> iterasi dilewati
            delta = objective(neighbor, vehicles) - current_value
            if delta > 0:
                current, current_value = neighbor, current_value + delta
            else:
                probability = math.exp(delta / T)
                probability_history[-1] = probability
                if delta < 0:
                    worse_considered += 1
                if probability > random.random():
                    current, current_value = neighbor, current_value + delta
                    if delta < 0:
                        worse_accepted += 1
                else:
                    stuck_count += 1

        value_history.append(current_value)
        T *= alpha
```

Perulangan berhenti ketika T < T_min atau iterasi mencapai maksimum. Pada tiap awal iterasi, temperatur dicatat dan 'probability_history' diberi nilai awal None. Nilai tersebut diganti ketika neighbor tidak lebih baik, sehingga nilai yang dimuat hanya yang $\Delta E \leq 0$. Jika 'random_neighbor()' mengembalikan 'None' (tidak ada tetangga valid), iterasi dilewati tanpa perubahan state.

Nilai ΔE dihitung sebagai selisih nilai objektif neighbor dan state saat ini. Jika $\Delta E > 0$, nrighbor langsung diterima. Jika $\Delta E \leq 0$, peluang penerimaan dihitung sebagai 'math.exp(delta / T)' dan dibandingkan melalui 'random.random()'. Neighbor diterima apabila peluang lebih besar dari bilangan acak tersebut. Pada $\Delta E = 0$ peluangnya tepat

1, sehingga *sideways move* selalu diterima: posisi atau orientasi kendaraan berubah tanpa mengubah nilai objektif.

Untuk tetangga dengan ΔE < 0 menambahkan 'worse_considered' . Jika diterima, 'worse_accepted' ikut bertambah, dan jika ditolak, 'stuck_count' bertambah karena state tidak berpindah meskipun tetangga yang lebih baik tidak ditemukan. Dengan demikian dalam laporan ini kondisi *stuck* didefinisikan dengan dua ukuran. Spesifikasi tidak mendefinisikan *stuck*, sehingga definisi ini merupakan interpretasi. Ukuran pertama adalah 'stuck_count'. Ukuran kedua menghitung percobaan worse accepted yang tidak berbuah yaitu tiap langkah turun yang diterima dan percobaan itu tidak pernah mencapai nilai yang lebih tinggi lagi sampai pencarian berhenti. Ukuran kedua dihitung dari `value_history` Sebagai pelengkap, pada analisis juga diperiksa apakah *final state* merupakan *local optimum* terhadap seluruh tetangganya.

Pengembalian Hasil dan Pengukuran Durasi

```
return {
        "initial_state": initial_state,
        "final_state": current, # mengembalikan state saat berhenti, bukan
best-so-far
        "final_score": current_value,
        "value_history": value_history,
        "temperature_history": temperature_history,
        "probability_history": probability_history,
        "stuck_count": stuck_count,
        "worse_considered": worse_considered,
        "worse_accepted": worse_accepted,
        "iterations": iterations,
        "duration_seconds": perf_counter() - start_time,
        "seed": seed,
        "T0": T0,
        "alpha": alpha,
        "T_min": T_min,
        "max_iter": max_iter
    }
```

Setelah loop selesai, fungsi mengembalikan *dictionary* berisi *initial state*, *final state*, nilai akhir, riwayat nilai, temperatur, dan peluang, penghitung stuck_count, worse_considered, dan worse_accepted, jumlah iterasi, durasi, serta parameter yang digunakan. State yang dikembalikan sebagai 'final_state' adalah state saat pencarian berhenti, bukan state terbaik yang pernah dijumpai selama pencarian (*best-so-far*), sehingga nilai akhir dapat lebih rendah daripada nilai tertinggi yang pernah dicapai selama pencarian.

Durasi dihitung dari selisih 'perf_counter()' saat hasil dikembalikan dengan waktu mulai yang dicatat setelah 'random.seed()'. Pengukuran mencakup pembentukan *initial state* dan seluruh iterasi pencarian, tetapi tidak mencakup pembuatan gambar dan penyimpanan berkas yang dilakukan pada fungsi 'SA_out()'.

Pengaturan Eksperimen

Eksperimen *Simulated Annealing* dilakukan sebanyak tiga kali menggunakan seed 1, 2, dan 3, dengan parameter yang sama pada setiap run (T0 = 800, alpha = 0,998, T_min = 1,0). Seluruh percobaan menggunakan data sepuluh kendaraan yang sama, dimensi geladak 5 × 15 satuan grid, dan kapasitas angkut maksimum 100 satuan berat. Karena 'random.seed()' dipanggil sebelum 'random_state()', dan *Hill Climbing* membentuk *initial state* melalui pemanggilan yang sama, *initial state* SA pada seed 1, 2, dan 3 sama dengan *initial state Hill Climbing* pada seed yang sama (nilai awal 850, 1050, dan 1050). Dengan demikian perbedaan hasil kedua algoritma bukan disebabkan oleh perbedaan *initial state*.

Pada setiap run dicatat *state* awal dan akhir, nilai objektif akhir, grafik nilai objektif dan e^(ΔE/T) terhadap iterasi, durasi pencarian, serta frekuensi *stuck*. Hasil lengkap tersimpan pada folder 'output/SA' pada repositori.

Rekap Hasil Eksperimen

Tabel berikut menyajikan hasil ketiga percobaan. Nilai objektif merupakan total shipping fee kendaraan yang dimuat. Kolom nilai tertinggi menunjukkan nilai terbesar yang pernah dimiliki state saat ini selama pencarian, sedangkan *stuck* adalah jumlah neighbor lebih buruk yang ditolak.

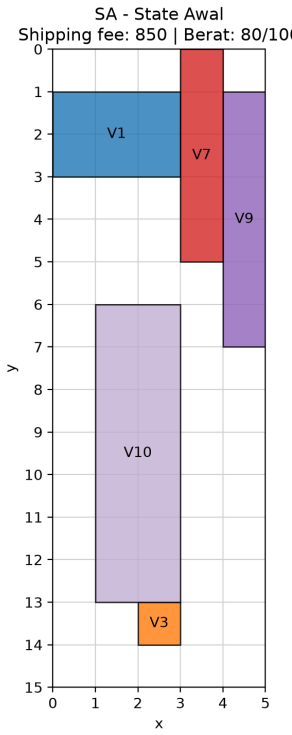
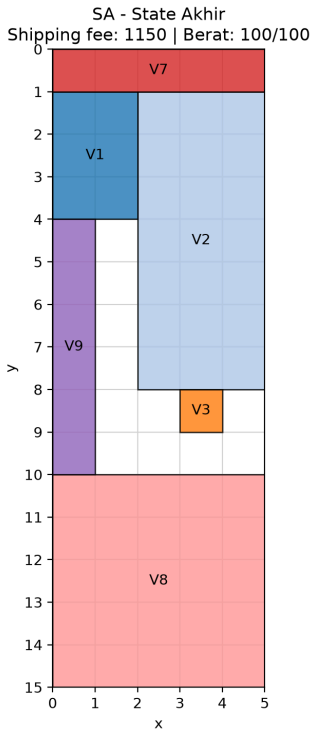
Rekap Hasil Eksperimen Simulated Annealing

Eksperimen	**Nilai Awal**	**Nilai Akhir**	**Nilai Tertinggi**	**Total Iterasi**	**Worse Accepted**	***Stuck***	**Durasi (detik)**
1	850	1150	1200 (iter. 288)	3339	55	340	19,051
2	1050	1150	1150 (iter. 1755)	3339	53	363	20,031
3	1050	1050	1150 (iter. 54)	3339	44	206	20,422

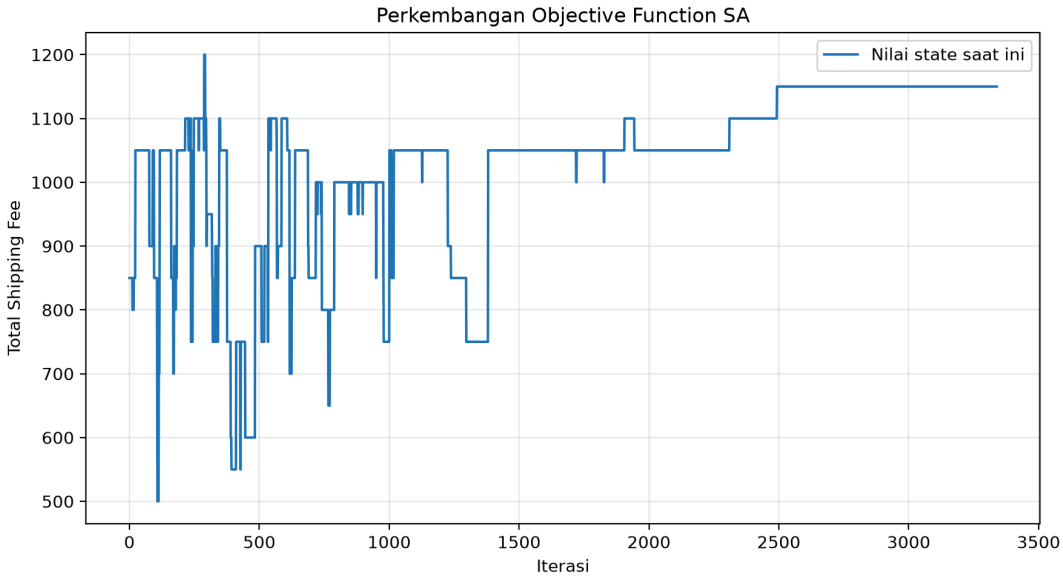
Seluruh temperatur mencapai total iterasi ke-3339 karena temperatur mencapai T_min. Nilai akhir berkisar antara 1050 dan 1150 dengan rata-rata 1116,67, sedangkan rata-rata durasi 19,834 detik. Nilai akhir tertinggi adalah 1150, diperoleh pada eksperimen 1 dan 2.

Eksperimen 1

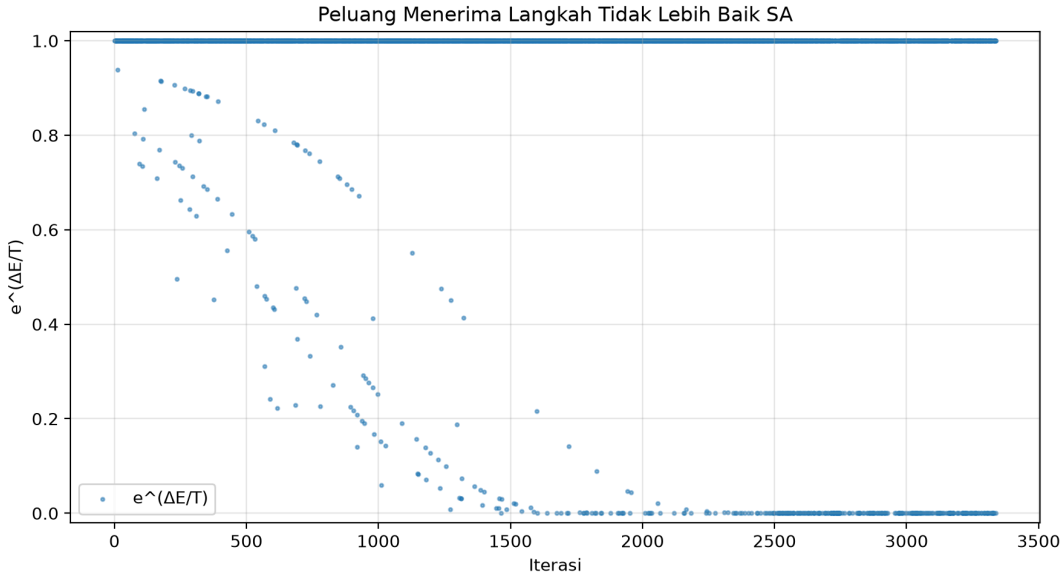
Eksperimen pertama dijalankan menggunakan seed 1. Gambar berikut memperlihatkan konfigurasi penempatan kendaraan sebelum dan sesudah pencarian, serta perkembangan nilai objektif dan peluang penerimaan selama proses berlangsung.

State Awal	State Akhir
 Tidak dimuat: V2, V4, V5, V6, V8	 Tidak dimuat: V4, V5, V6, V10

Perkembangan Nilai Objektif:



Peluang Penerimaan e^(ΔE/T) terhadap Iterasi:

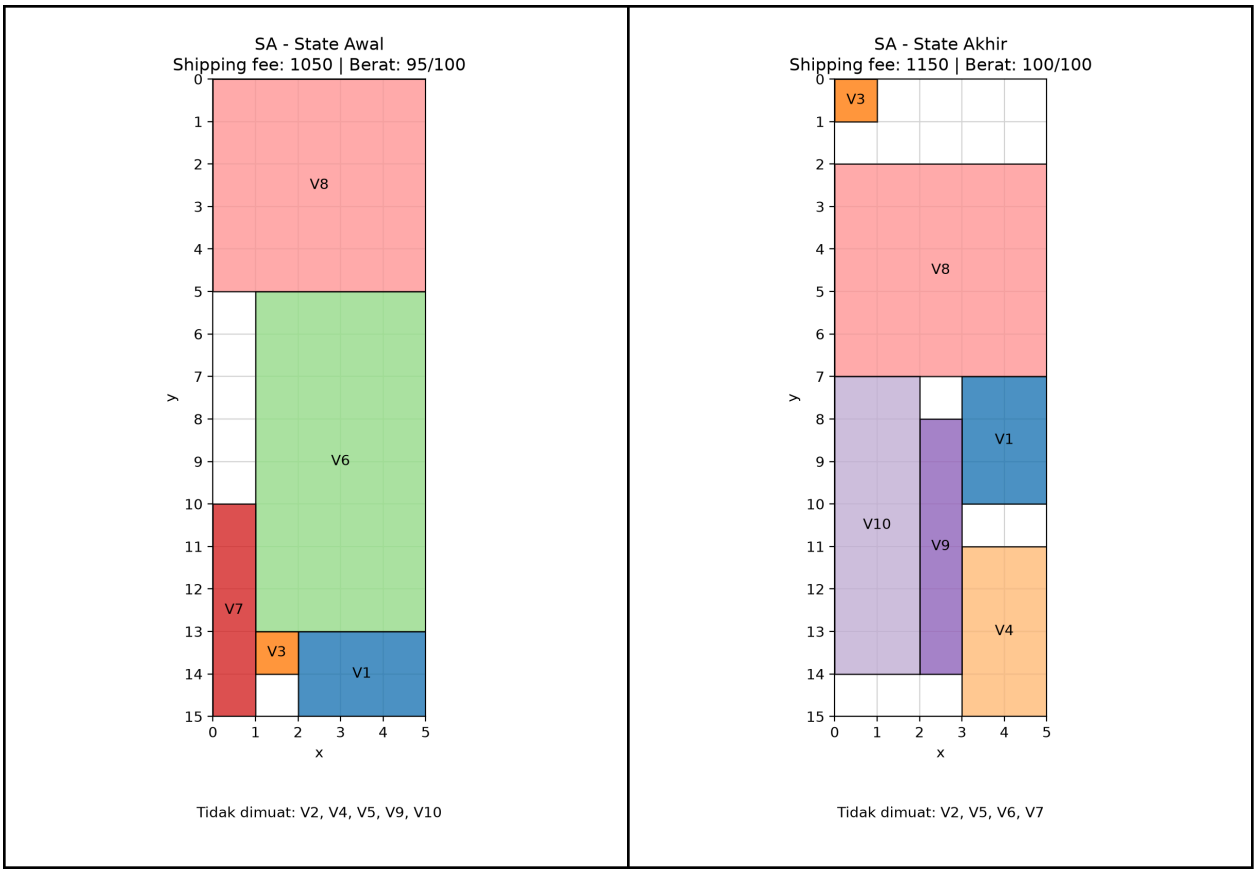


State awal pada eksperimen pertama memiliki nilai objektif sebesar 850, terendah di antara ketiga run. Nilai objektif turun hingga 500 pada sekitar iterasi ke-110, kemudian naik kembali, dan mencapai nilai tertinggi 1200 pada iterasi ke-288. Nilai 1200 tersebut tidak dipertahankan karena pada temperatur setinggi itu langkah yang lebih buruk masih mudah diterima. State terakhir bernilai 1150 dicapai pada iterasi ke-2493 (T ≈ 5,5) dan bertahan hingga pencarian berhenti. Total *shipping fee* meningkat sebesar 300 atau sekitar 35,29% dibandingkan nilai awal, tetapi 50 lebih rendah daripada nilai tertinggi yang pernah dijumpai.

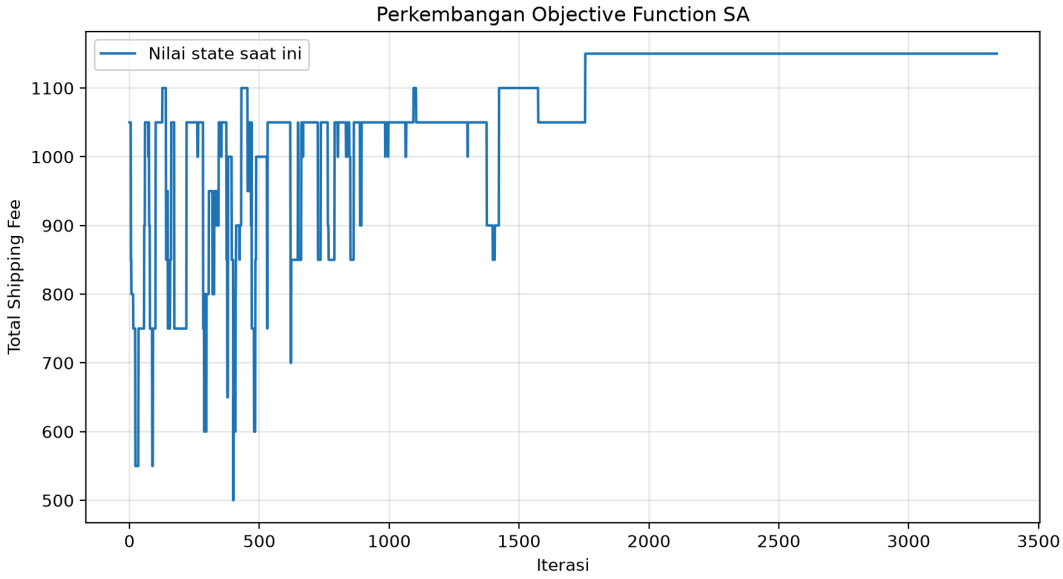
Eksperimen 2

Eksperimen kedua dijalankan menggunakan seed 2. Visualisasi berikut memperlihatkan state awal dan akhir, serta perkembangan nilai objektif dan peluang penerimaan.

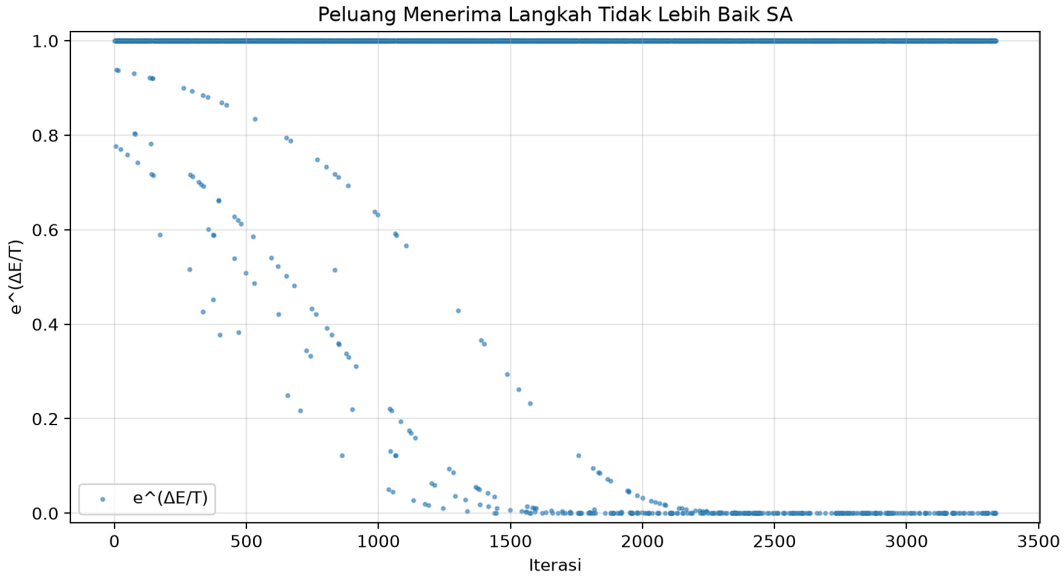
State Awal	State Akhir



Perkembangan Nilai Objektif:



Peluang Penerimaan e^(ΔE/T) terhadap Iterasi:

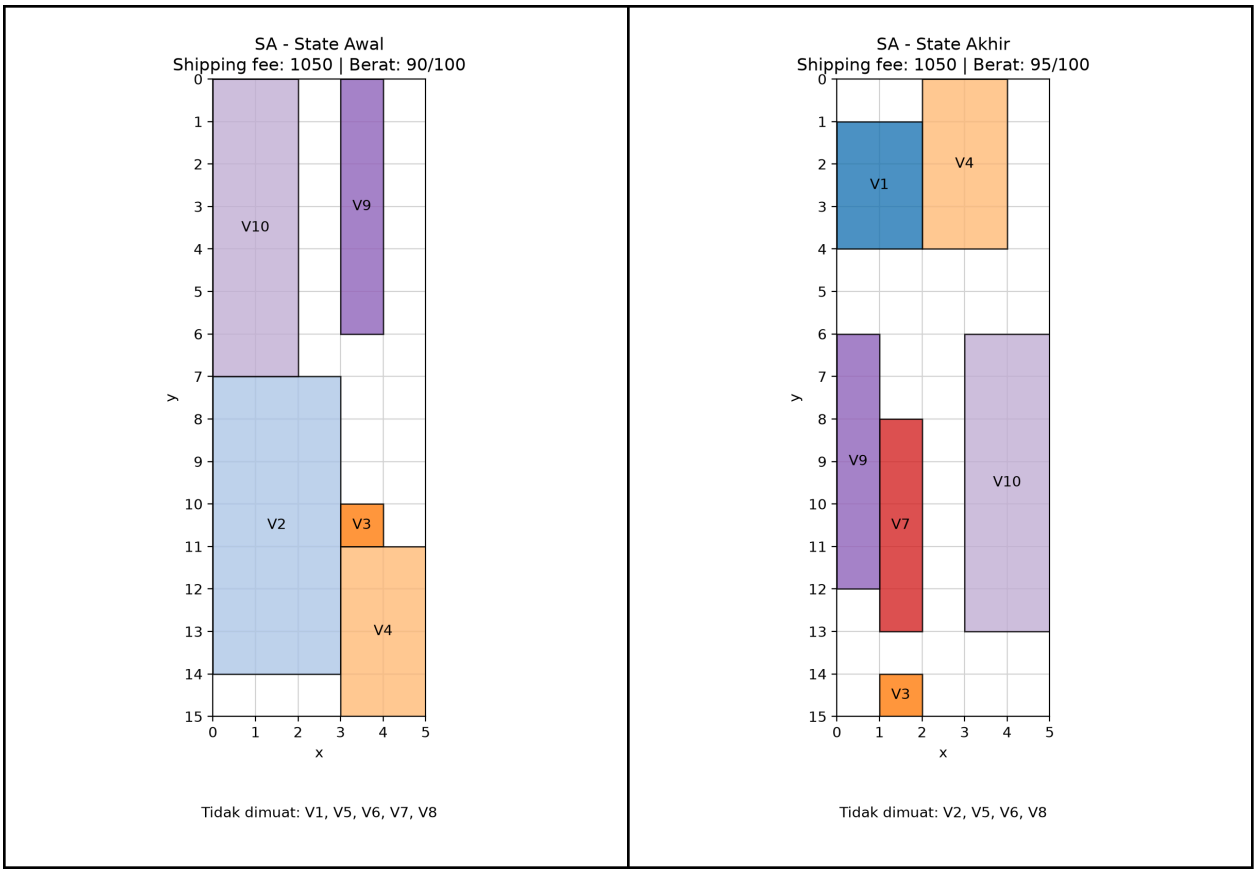


State awal pada eksperimen kedua bernilai 1050. Nilai objektif bergerak naik turun pada rentang 500 hingga 1100 selama fase temperatur tinggi, kemudian naik menjadi 1100 dan akhirnya mencapai 1150 pada iterasi ke-1755. Kenaikan ke 1150 didahului langkah lebih buruk yang diterima (iterasi ke-1574, 1100 menjadi 1050, T ≈ 34), yang menunjukkan mekanisme SA melewati state yang lebih buruk untuk mencapai state yang lebih baik. Nilai 1150 tersebut merupakan nilai tertinggi sekaligus nilai akhir, dan tidak berubah hingga akhir. Total *shipping fee* meningkat sebesar 100 atau sekitar 9,52% dibandingkan nilai awal.

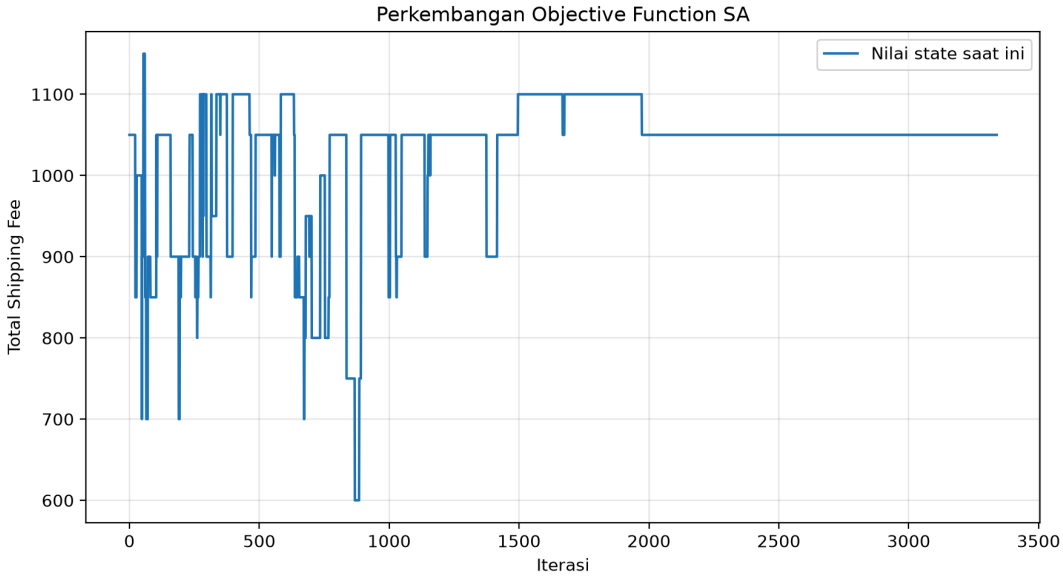
Eksperimen 3

Eksperimen ketiga dijalankan menggunakan seed 3. Visualisasi berikut memperlihatkan konfigurasi awal dan akhir penempatan kendaraan, serta perkembangan nilai objektif dan peluang penerimaan.

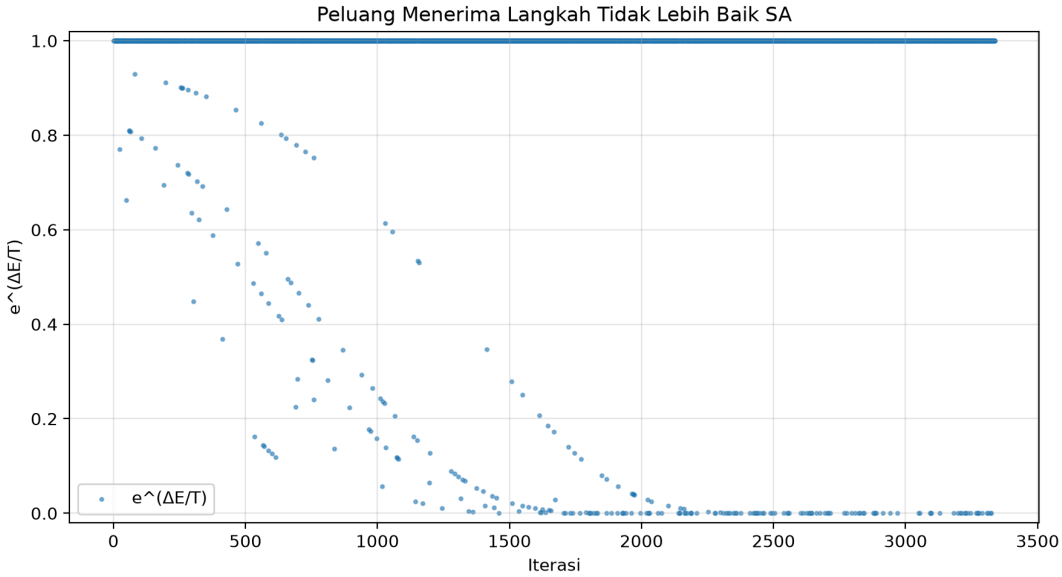
State Awal	State Akhir



Perkembangan Nilai Objektif:



Peluang Penerimaan e^(ΔE/T) terhadap Iterasi:



State awal pada eksperimen ketiga bernilai 1050 dan nilai akhirnya juga 1050, sehingga tidak ada peningkatan dibandingkan nilai awal. Nilai tertinggi 1150 justru dicapai sangat awal, pada iterasi ke-54 saat T ≈ 720, ketika pencarian masih mendekati *random walk*. Nilai tersebut tidak dipertahankan. Perubahan nilai objektif terakhir terjadi pada iterasi ke-1973, ketika nilai turun dari 1100 menjadi 1050 melalui *worse accepted* dengan peluang sekitar 3,9%. Setelah itu 1366 iterasi terakhir tidak mengubah nilai objektif sama sekali, sehingga nilai akhir lebih rendah daripada nilai yang sempat dipegang sebelumnya.

Hasil dan Analisis Eksperimen Simulated Annealing

Nilai tertinggi secara berat dan luas adalah 1300 (V1, V2, V4, V8, V10; berat 100, luas 74), tetapi himpunan tersebut tidak dapat ditata tanpa tumpang tindih. Optimum global adalah 1250. *Final state* SA bernilai 1150, 1150, dan 1050, atau 92,0%, 92,0%, dan 84,0% dari optimum global, dengan rata-rata 1116,67 (89,33%). Selisih terhadap optimum adalah 100, 100, dan 200 (rata-rata 133,33). Tidak ada run yang mencapai 1250, dan nilai tertinggi yang pernah dijumpai selama pencarian pun hanya 1200, 1150, dan 1150. Nilai 1200 pada run 1 (iterasi ke-288, T ≈ 450) dan 1150 pada run 3

(iterasi ke-54, T ≈ 720) dicapai pada temperatur tinggi dan tidak dipertahankan karena SA mengembalikan state terakhir, bukan *best-so-far*.

Terdapat tiga penyebab yang konsisten dengan data. Pertama, ketiga *final state* merupakan *local optimum* terhadap seluruh tetangganya, tidak ada satu pun neighbor yang bernilai lebih tinggi, stuck. Kedua, lanskap pencarian didominasi *plateau*. Nilai objektif bergantung pada himpunan kendaraan yang dimuat, sedangkan sebagian besar perubahan (posisi dan orientasi) tidak mengubah himpunan tersebut. Akibatnya 86,1% hingga 91,2% iterasi merupakan langkah dengan ΔE = 0 dan nilai objektif tidak berubah banyak. penataan yang memungkinkan penambahan kendaraan bernilai tinggi jarang ditemukan oleh perpindahan acak. Ketiga, pendinginan membuat pencarian membeku sebelum akhir. Sebelum iterasu berakhir sudah tidak ada satu pun langkah lebih buruk yang dapat diterima (peluang dan temperatur sudah terlalu rendah), SA hanya dapat mempertahankan atau menaikkan nilai, sehingga berperilaku seperti *Hill Climbing* dengan *sideways move*.

Distribusi Iterasi dan Frekuensi Stuck

Tabel berikut menunjukkan klasifikasi seluruh iterasi berdasarkan nilai ΔE

Eksperimen	**Lebih baik (ΔE > 0)**	**Sama (ΔE = 0)**	**Lebih buruk diterima**	**Lebih buruk ditolak (*stuck*)**	**Total iterasi**
1	48	2896	55	340	3339
2	49	2874	53	363	3339
3	44	3045	44	206	3339

Jika iterasi dibagi kedalam tiga fase, didapatkan distribusi *worse accepted* seperti pada tabel berikut

Fase	Rentang T	Eksperimen 1	Eksperimen 2	Eksperimen 3
Iterasi 1–1113	800 → 86,3	48/87	49/86	39/78
Iterasi 1114–2226	86,2 → 9,3	7/74	4/135	5/94
Iterasi 2227–3339	9,3 → 1,0	0/234	0/195	0/78

Frekuensi *stuck* adalah 340, 363, dan 206 kali dari seluruh iterasi. Dari seluruh langkah yang lebih buruk, 86,1%, 87,3%, dan 82,4% ditolak. Penolakan paling banyak terjadi pada fase akhir. Seluruh langkah yang dievaluasi pada fase ketiga ditolak. Selain itu, ketiga *final state* merupakan *local optimum*, sehingga pada akhir pencarian SA pada setiap run terjebak pada *local optima* yang tidak berhasil ditinggalkan. Eksperiemn 3 memiliki *stuck* paling sedikit tetapi hasil paling rendah. Hal ini menunjukkan bahwa jumlah *stuck* bukan ukuran kualitas hasil.

Percobaan yang Tidak Berbuah

Ukuran kedua menghitung percobaan worse accepted lalu memisahkan yang berhasil (nilai di atas titik asal tercapai kemudian) dari yang tidak berbuah. Titik asal adalah nilai sebelum turun, *yang belum tentu merupakan local optimum*.

Tabel Percobaan Kabur Simulated Annealing

Eksperimen	Percobaan Worse Accepted	Berhasil	Tidak berbuah	Tidak pernah kembali ke nilai asal
1	55	54	1	1
2	53	53	0	0
3	44	33	11	2

Secara total 12 dari 152 percobaan tidak berbuah. Pada run 1 dan 2 hampir semua langkah turun akhirnya diikuti nilai yang lebih tinggi, tetapi angka ini kurang bermakna karena sebagian besar langkah turun terjadi di awal dari nilai yang masih rendah, sehingga nilai yang lebih tinggi mudah ditemukan lagi tanpa bergantung pada langkah turun itu. Run 3 paling banyak gagal. SA berulang kali turun dari nilai *lokal optima* tanpa pernah menemukan yang lebih tinggi.

Pengaruh Temperatur terhadap Peluang Penerimaan

Grafik e^(ΔE/T) pada tiap eksperimen memperlihatkan dua pola. Pertama, deretan titik pada nilai 1,0 berasal dari langkah ΔE = 0 (nilai sama) yang selalu diterima. Jumlahnya sangat banyak, yaitu 86% sampai 91% iterasi, sehingga mendominasi grafik. Kedua, titik-titik di bawahnya turun seiring waktu karena temperatur makin kecil, sehingga langkah yang lebih buruk makin sulit diterima.

Sebagai contoh, langkah yang menurunkan nilai 50 punya peluang diterima 0,94 pada awal (T = 800), tetapi hanya sekitar 0,08 pada T = 20 dan hampir nol pada T = 10. Sejak sekitar iterasi 1600, sebagian besar titik langkah lebih buruk sudah mendekati nol, artinya SA hampir tidak lagi menerima langkah yang lebih buruk.

Konsistensi Hasil

Nilai akhir ketiga run adalah 1150, 1150, dan 1050, atau selisih 100 dari yang tertinggi ke terendah. Dua dari tiga run menghasilkan nilai yang sama. Peningkatan terhadap state awal berbeda di tiap run (+300, +100, dan 0). Run 1 yang mulai dari nilai terendah (850) justru berakhir di nilai tertinggi bersama run 2, karena di fase awal SA mudah berpindah ke state yang lebih buruk sehingga state awal tidak terlalu menentukan. Namun dengan hanya tiga run, kesimpulan ini masih belum dapat diuji secara statistik 100%.

Durasi Pencarian

Durasi pencarian 19,051; 20,031; dan 20,422 detik (rata-rata sekitar 19,83 detik). Selisih anatar run kecil karena jumlah iterasinya sama. Waktu per iterasi sekitar 6 miliseken karena 'random_neighbor()' memvalidasi seluruh neighbor tiap iterasi, walaupun hanya satu neighbor yang dibutuhkan tiap step. Jadi durasi yang tinggi diakibatkan implementasi pemilihan neighbor, bukan sifat bawaan SA.

Genetic Algorithm

Pendekatan melalui *Genetic Algorithm* dilakukan dengan ketentuan berikut:

- Analisis akan menggunakan 3 variansi untuk masing-masing *population* dan *iteration* (jumlah populasi, dan jumlah generasi) untuk 6 eksperimen berbeda
- Parameter *mutation* ditetapkan dengan value 0.1 (peluang mutasi 10% per kendaraan)
- Setiap eksperimen dijalankan sebanyak 3 kali
- Masing-masing eksperimen memiliki *initial state* yang berbeda
- Rekap hasil algoritma disimpan dalam bentuk JSON

Penjelasan Implementasi

Algoritma Genetic Algorithm mencari maksimum dari objective dengan cara: pembentukan populasi awal, seleksi *parent*, *crossover*, dan mutasi sepanjang sejumlah generasi/iterasi. Setiap state direpresentasi sebagai Map/Dictionary yang berisi id kendaraan sebagai key, dan data *placement* kendaraan sebagai value sebagai gen. Populasi direpresentasikan sebagai list dari complete state yang acak dan akan diacak kembali untuk setiaap generasi kecuali dengan anak hasil *crossover*.

Genetic Algorithm memiliki alur sebagai berikut: melakukan inisialisasi populasi berdasarkan setting *population_size*; inisialisasi list berisi skor dari masing-masing *state* dalam populasi yang disimpan dalam list *max_history* dan *average_history*; 2

state akan diseleksi melalui fungsi *select_parents* dimana kedua state tersebut akan dilakukan fungsi *crossover* untuk mengenerate anak sebanyak 2 kali untuk membuat 2 anak; Kedua anak tersebut akan dilakukan mutasi melalui fungsi *mutate*; Populasi baru yang diinisialisasi pada awal loop (kosong) akan menambah kedua anak tersebut (anak pertama saja apabila jumlah/ukuran populasi tidak cukup); Skor dari generasi/iterasi ini akan disimpan ke dalam *max_history dan average_history*; Proses ini mulai dari seleksi state hingga ini akan diulang berdasarkan setting iterasi yang digunakan sebagai bahan analisis eksperimen; Best state lalu menyimpan state dengan nilai objective maksimal dari **generasi terakhir**; Fungsi lalu akan mereturn map/dictionary berisi data: populasi awal, populasi akhir, best state, skor objective final, skor maksimal pencarian (per generasi populasi), skor rata-rata pencarian (per generasi populasi), jumlah iterasi, *mutation_rate*, dan durasi pencarian (dalam detik).

```
def genetic_algorithm(vehicles, ship, population_size, iterations,
mutation_rate):
    start_time = perf_counter()
    population = generate_population(population_size, vehicles, ship)
    initial_population = [s.copy() for s in population]
    scores = [objective(s, vehicles) for s in population]
    max_history = [max(scores)]
    average_history = [sum(scores) / len(scores)]

    for _ in range(iterations):
        new_population = []

        while len(new_population) < population_size:
            parent1, parent2 = select_parents(population, vehicles)
            child1 = crossover(parent1, parent2, vehicles, ship)
            child2 = crossover(parent2, parent1, vehicles, ship)

            child1 = mutate(child1, vehicles, ship, mutation_rate)
            child2 = mutate(child2, vehicles, ship, mutation_rate)
```

```
            new_population.append(child1)
            if len(new_population) < population_size:
                new_population.append(child2)

        population = new_population
        scores = [objective(s, vehicles) for s in population]
        max_history.append(max(scores))
        average_history.append(sum(scores) / len(scores))

    best_state = max(population, key=lambda s: objective(s, vehicles))
    return {
    "initial_population": initial_population,
    "final_population": population,
    "final_state": best_state,
    "final_score": objective(best_state, vehicles),
    "max_history": max_history,
    "average_history": average_history,
    "population_size": population_size,
    "iterations": iterations,
    "mutation_rate": mutation_rate,
    "duration_seconds": perf_counter() - start_time
}
```

Generasi populasi diimplementasi sebagai pembuatan state acak berulang. Hasil dari state acak ini dimasukan ke dalam list kosong dan diulang sebanyak nilai dari *population* yang diubah variansinya sebagai bahan analisis eksperimen.

```
def generate_population(size, vehicles, ship):
    population = []
    for i in range(size):
        state = random_state(vehicles, ship)
        population.append(state)
    return population
```

Fitness function menggunakan objective untuk menghitung kemungkinan seleksi state sebagai *parent* pada fungsi *crossover*. Apabila pada fungsi select parents skor fitness adalah 0, maka akan mereturn pasangan (tuple) state yang acak dari populasi tanpa perhitungan bobot nilai fitness.

```
def select_parents(population, vehicles):
    if not population:
        raise ValueError("populasi tak boleh kosong")
    fitness_scores = [objective(state, vehicles) for state in population]
    if sum(fitness_scores) == 0:
        return random.choices(population, k=2)
    return random.choices(population, weights=fitness_scores, k=2)
```

Crossover diimplementasikan dengan menginput 2 state yang didapatkan dengan fungsi *select_parents* lalu memasukan data *placement* dari kedua *parent* berdasarkan variabel *cut* yaitu nilai random antara 1 hingga (jumlah kendaraan-1) untuk mengabungkan data kedua *parent*. Sebagai fallback, apabila state akhir dari anak tidak valid, *crossover* akan mereturn salah satu state *parent* sebagai anak.

```
def crossover(parent1, parent2, vehicles, ship):
    ids = list(vehicles)
    if len(ids) < 2:
        return parent1.copy()
    cut = random.randint(1, len(ids) - 1)
    parent1 = parent1.copy()
    parent2 = parent2.copy()
    child = State({id: Placement() for id in vehicles})
    for index, id in enumerate(ids):
        if index < cut:
            child.placements[id] = parent1.placements[id]
        else:
            child.placements[id] = parent2.placements[id]
    if not is_valid(child, vehicles, ship):
        return random.choice([parent1, parent2]).copy()
```

```
    return child
```

Terdapat mutation rate yaitu 0.1 atau peluang 10% (bisa diubah namun tidak digunakan dalam eksperimen) dimana di awal fungsi apabila fungsi random nilainya lebih rendah akan melakukan mutasi pada state yang dimasukan. Fungsi *mutate* akan memilih kendaraan acak, dan lalu akan melakukan "lemparan koin" untuk menentukan bagaimana mutasi dilakukan. Bila True, placement dari kendaraan tersebut akan di-set menjadi loaded (pasti akan dimasukan ke kapal), dan data-data koordinat x, y, dan variabel rotated akan ditentukan secara acak. Namun bila False, placement akan di-set menjadi unloaded (tidak dimasukan ke kapal). Sebagai fallback, apabila state akhir hasil mutasi tidak valid, maka akan me-return state awal tanpa adanya perubahan.

```
def mutate(state, vehicles, ship, mutation_rate=0.1):
    temp_state = state.copy()
    if random.random() >= mutation_rate or not vehicles:
        return temp_state
    id = random.choice(list(vehicles))
    if random.choice([True, False]):
        p = Placement(
            loaded=True,
            rotated=random.choice([False, True])
        )
        w, l = get_dimensions(vehicles[id], p)
        if w > ship.width or l > ship.length:
            return temp_state
        p.x = random.randint(0, ship.width - w)
        p.y = random.randint(0, ship.length - l)
    else:
        p = Placement(
            loaded=False,
            rotated=temp_state.placements[id].rotated
        )
    temp_state.placements[id] = p
```

```
    if not is_valid(temp_state, vehicles, ship):
        return state.copy()
    return temp_state
```

Fungsi GA_out menjalankan Genetic Algorithm sebanyak tiga kali untuk satu konfigurasi parameter. Hasil setiap run dikonversi menjadi dictionary/map yang disimpan dalam format JSON. Fungsi ini juga menyimpan visualisasi individu terbaik populasi awal dan generasi terakhir untuk setiap run. Selanjutnya, fungsi plot_ga_history menggabungkan grafik nilai objective maksimum dan rata-rata populasi dari ketiga run ke dalam satu gambar. Hasil dari gambar, grafik, dan file JSON disimpan pada folder output/GA di repo.

```
def GA_out(vehicles, ship, population_size: int, iterations: int,
mutation_rate: float):
    def to_dict(state):
        data = {}
        for id, p in state.placements.items():
            data[id] = {
                "loaded": p.loaded,
                "x": p.x,
                "y": p.y,
                "rotated": p.rotated
            }
        return data

    outputs = []
    output_path = Path(__file__).resolve().parents[2] / "output" / "GA"
    output_path.mkdir(parents=True, exist_ok=True)
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S_%f")
    base_name = f"GA_pop{population_size}_gen{iterations}_{timestamp}"

    for i in range(3):
        result = genetic_algorithm(
            vehicles, ship, population_size, iterations, mutation_rate
```

```
        )
        output = {
            "initial_population": [to_dict(s) for s in
result["initial_population"]],
            "final_population": [to_dict(s) for s in
result["final_population"]],
            "final_state": to_dict(result["final_state"]),
            "final_score": result["final_score"],
            "max_history": result["max_history"],
            "average_history": result["average_history"],
            "population_size": result["population_size"],
            "iterations": result["iterations"],
            "mutation_rate": result["mutation_rate"],
            "duration_seconds": result["duration_seconds"]
        }
        outputs.append(output)
        run_name = f"{base_name}_run{i + 1}"
        file_path = output_path / f"{run_name}.json"
        with file_path.open("w", encoding="utf-8") as file:
            json.dump(output, file, indent=4)
        initial_state = max(result["initial_population"], key=lambda s:
objective(s, vehicles))
        initial_result = plot_state(initial_state, vehicles, ship, f"GA Run {i
+ 1} - State Terbaik | Populasi Awal \n Populasi: {population_size} |
Generasi: {iterations}")
        final_result = plot_state(result["final_state"], vehicles, ship, f"GA
Run {i + 1} - State Terbaik | Generasi Terakhir \n Populasi: {population_size}
| Generasi: {iterations}")
        initial_result.savefig(output_path / f"{run_name}_initial.png",
dpi=200, bbox_inches="tight")
        final_result.savefig(output_path / f"{run_name}_final.png", dpi=200,
bbox_inches="tight")
        plt.close(initial_result)
        plt.close(final_result)

    history_result = plot_ga_history(outputs)
    history_result.savefig(
```

```
        output_path / f"{base_name}_history.png",
        dpi=200, bbox_inches="tight"
    )

    plt.close(history_result)
    return outputs
```

```
def plot_ga_history(results):
    fig, ax = plt.subplots(figsize=(10, 6))

    for index, result in enumerate(results):
        iterations = range(len(result["max_history"]))
        line, = ax.plot(
            iterations,
            result["max_history"],
            label=f"Run {index + 1} maksimum"
        )
        ax.plot(
            iterations,
            result["average_history"],
            color=line.get_color(),
            linestyle="--",
            label=f"Run {index + 1} rata-rata"
        )

    ax.set_title(f"Perkembangan Objective Function GA | Populasi:
{results[0]['population_size']} | Generasi: {results[0]['iterations']}")
    ax.set_xlabel("Iterasi")
    ax.set_ylabel("Total Shipping Fee")
    ax.grid(True, alpha=0.3)
    ax.legend(ncol=2)
    fig.tight_layout()
    return fig
```

Tabel Parameter Eksperimen Genetic Algorithm

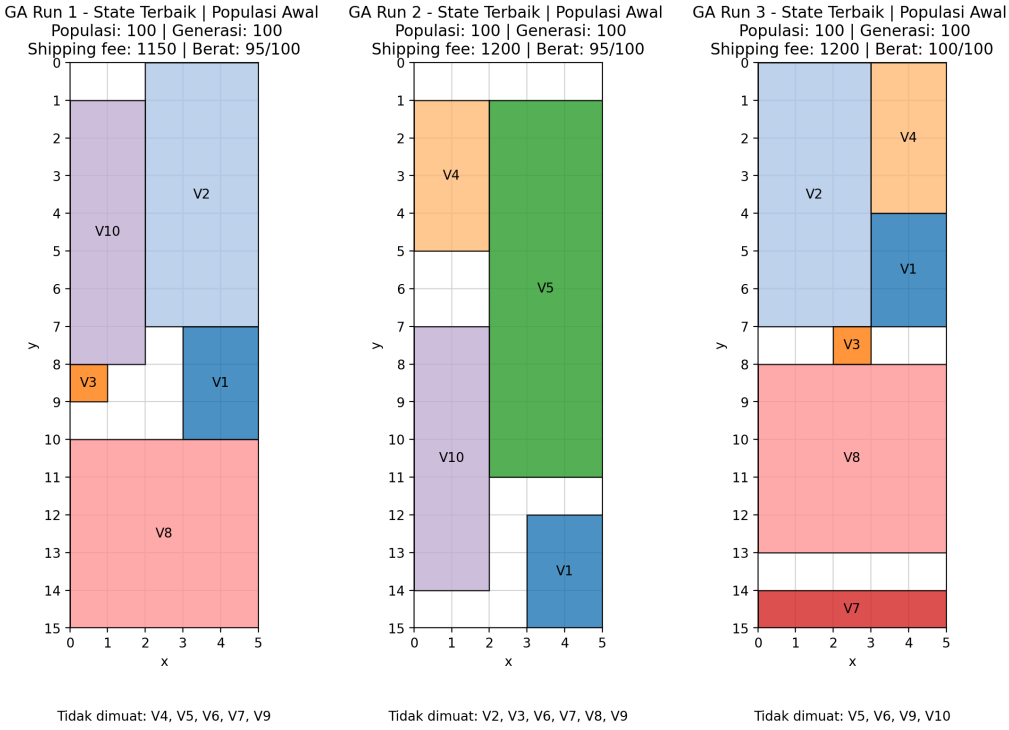
Exp. No.	*Population*	*Iteration/Generation*
Control	100	100
1	100	10
2	100	50
3	100	200
4	10	100
5	50	100
6	200	100

Eksperimen Kontrol

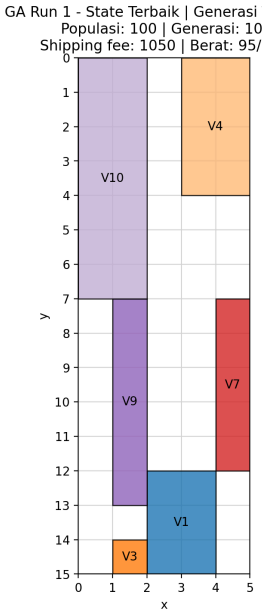
Data Eksperimen:

Run	Maks. Inisial	Skor Final	Maks. Pencarian	Durasi (detik)
1	1150	1050	1150	0.587
2	1200	1200	1200	0.487
3	1200	1100	1200	1.636

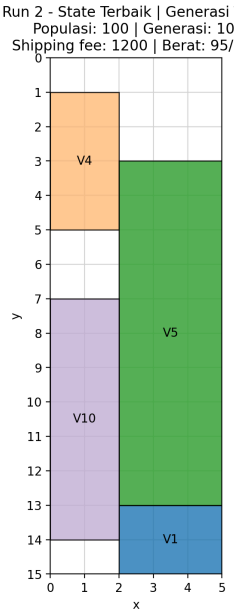
Initial State:



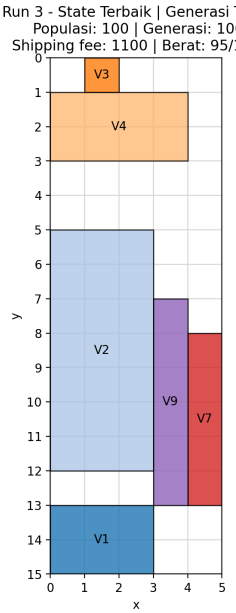
Final State:



Tidak dimuat: V2, V5, V6, V8

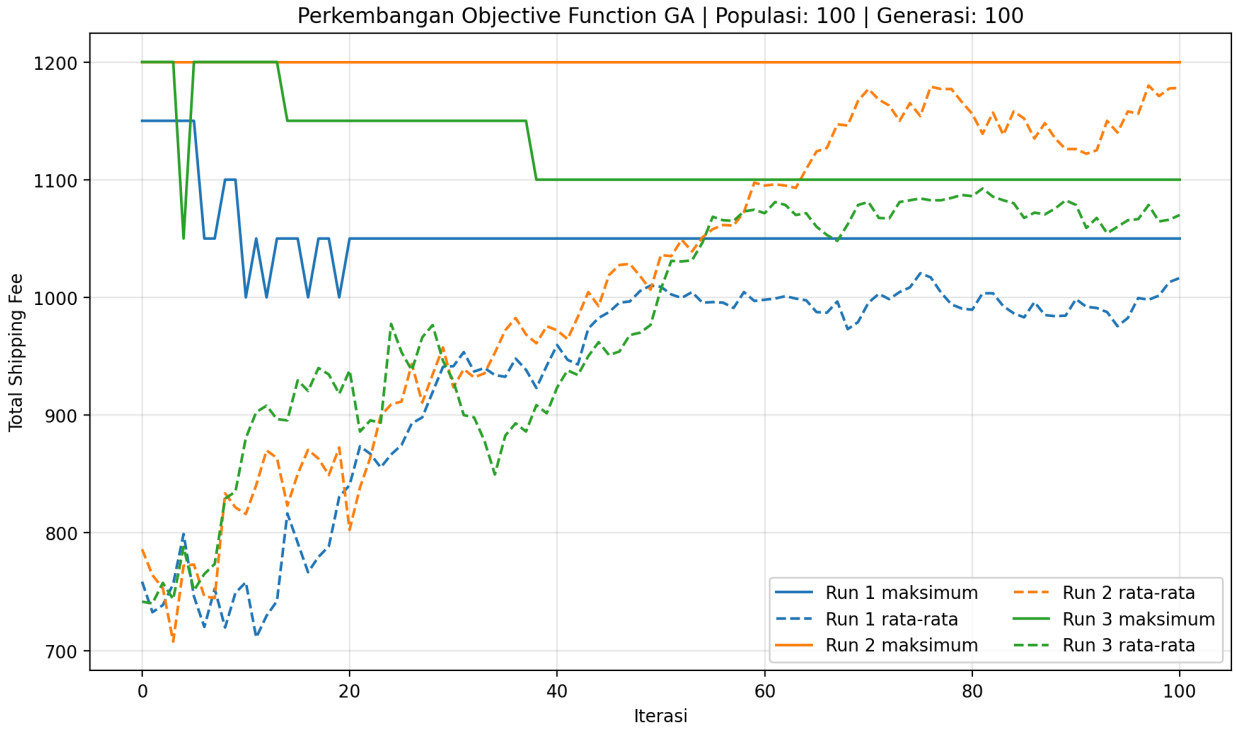


Tidak dimuat: V2, V3, V6, V7, V8, V9



Tidak dimuat: V5, V6, V8, V10

Function Objective Progression Histogram:

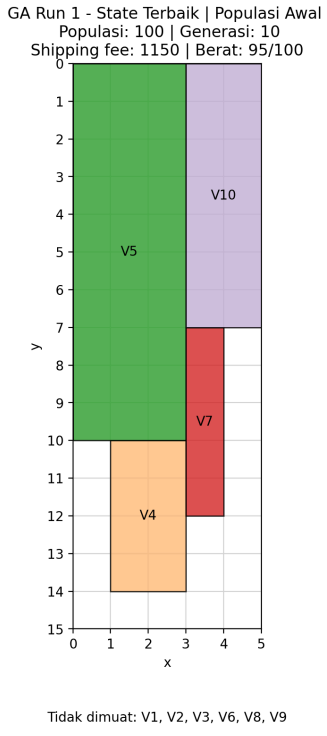


Eksperimen 1

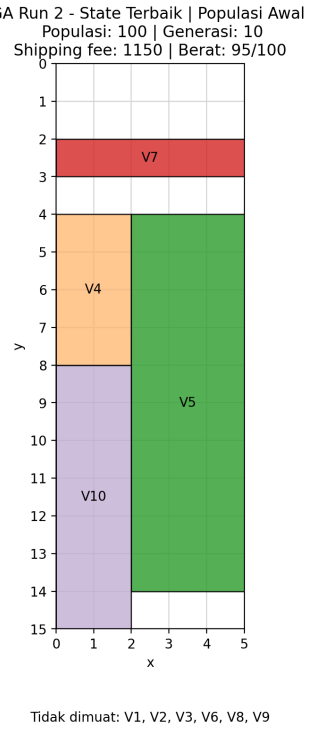
Data Eksperimen:

Run	Maks. Inisial	Skor Final	Maks. Pencarian	Durasi (detik)
1	1150	1200	1200	0.300
2	1150	1050	1150	0.371
3	1150	950	1150	0.379

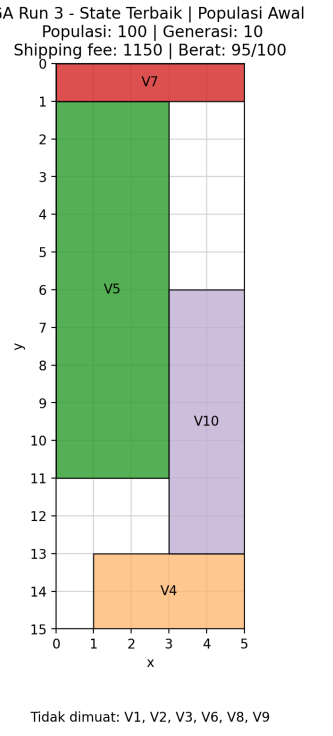
Initial State:



Tidak dimuat: V1, V2, V3, V6, V8, V9

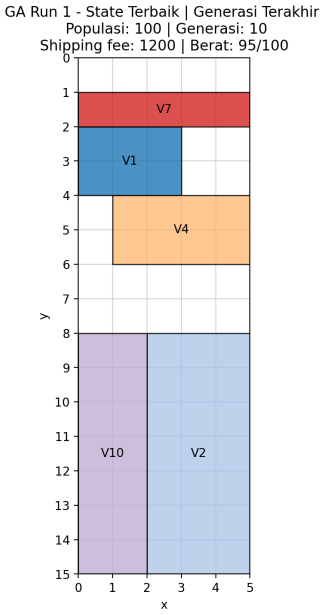


Tidak dimuat: V1, V2, V3, V6, V8, V9

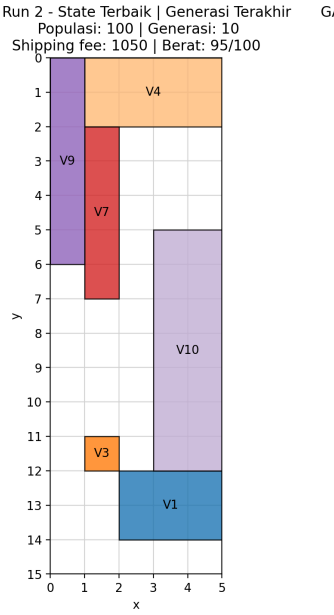


Tidak dimuat: V1, V2, V3, V6, V8, V9

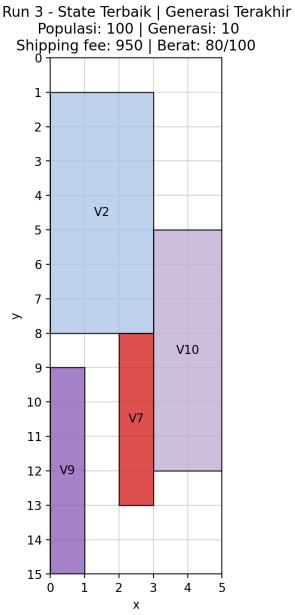
Final State:



Tidak dimuat: V3, V5, V6, V8, V9

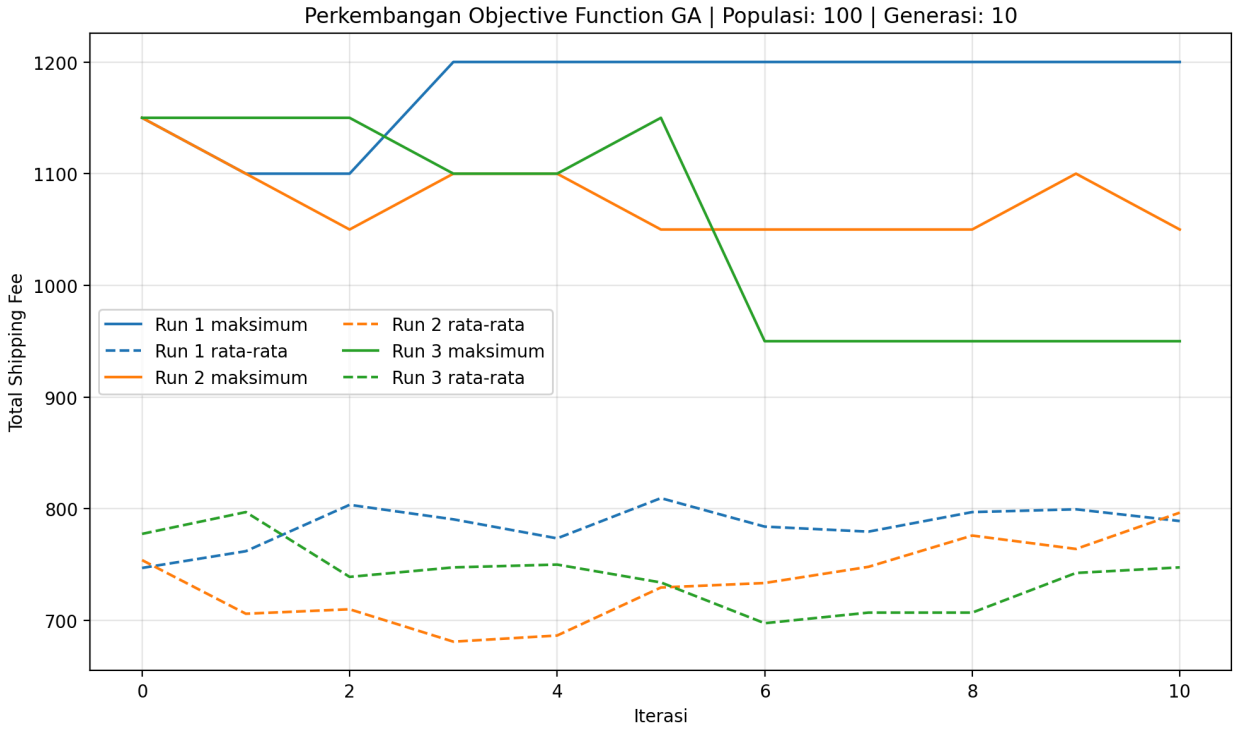


Tidak dimuat: V2, V5, V6, V8



Tidak dimuat: V1, V3, V4, V5, V6, V8

Function Objective Progression Histogram:

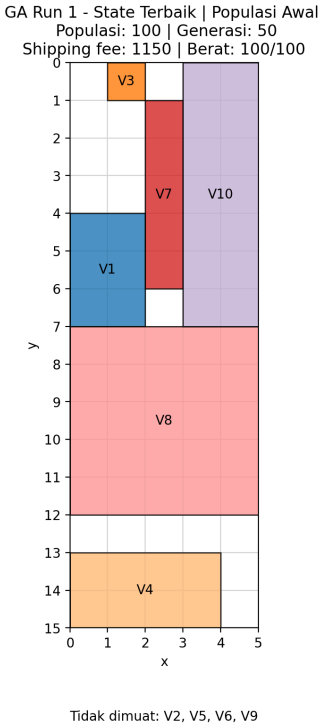


Eksperimen 2

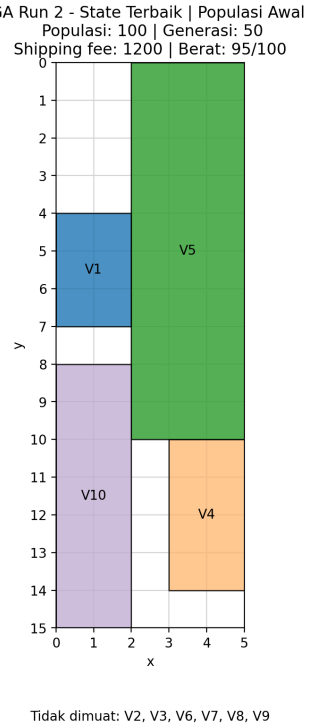
Data Eksperimen:

Run	Maks. Inisial	Skor Final	Maks. Pencarian	Durasi (detik)
1	1150	1150	1150	1.042
2	1200	1150	1200	0.851
3	1200	1200	1200	1.047

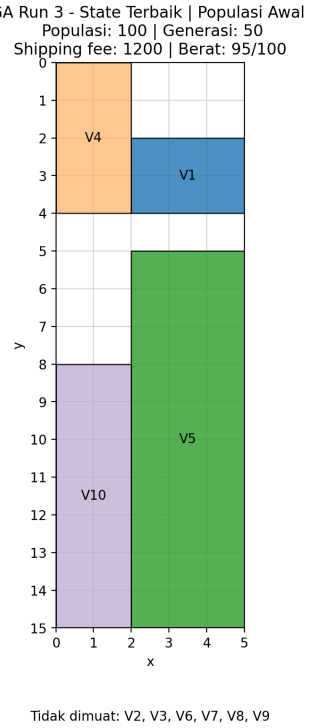
Initial State:



Tidak dimuat: V2, V5, V6, V9

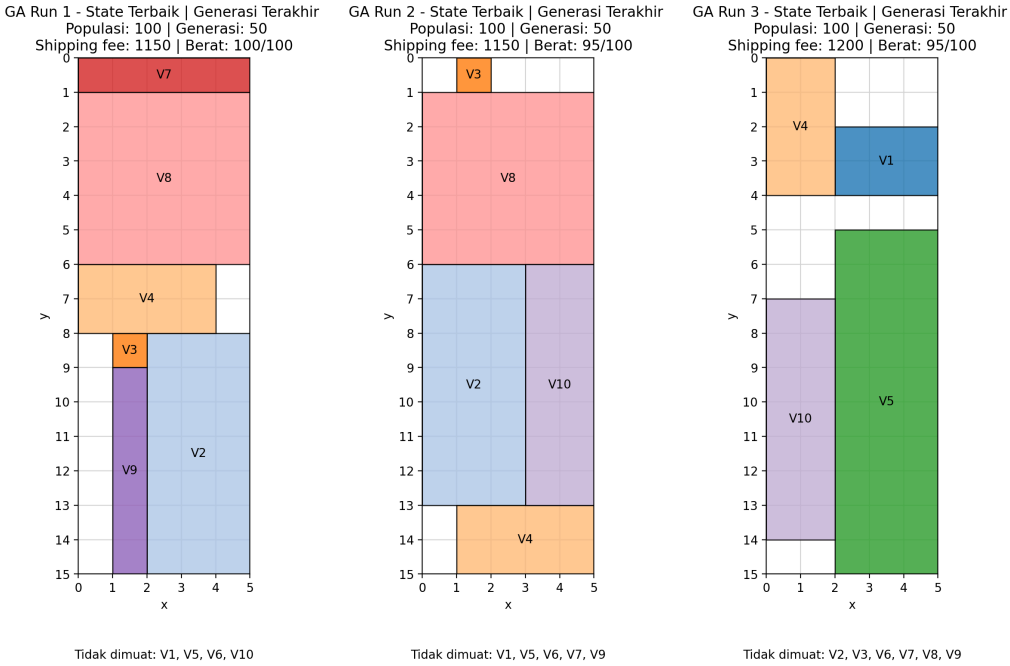


Tidak dimuat: V2, V3, V6, V7, V8, V9

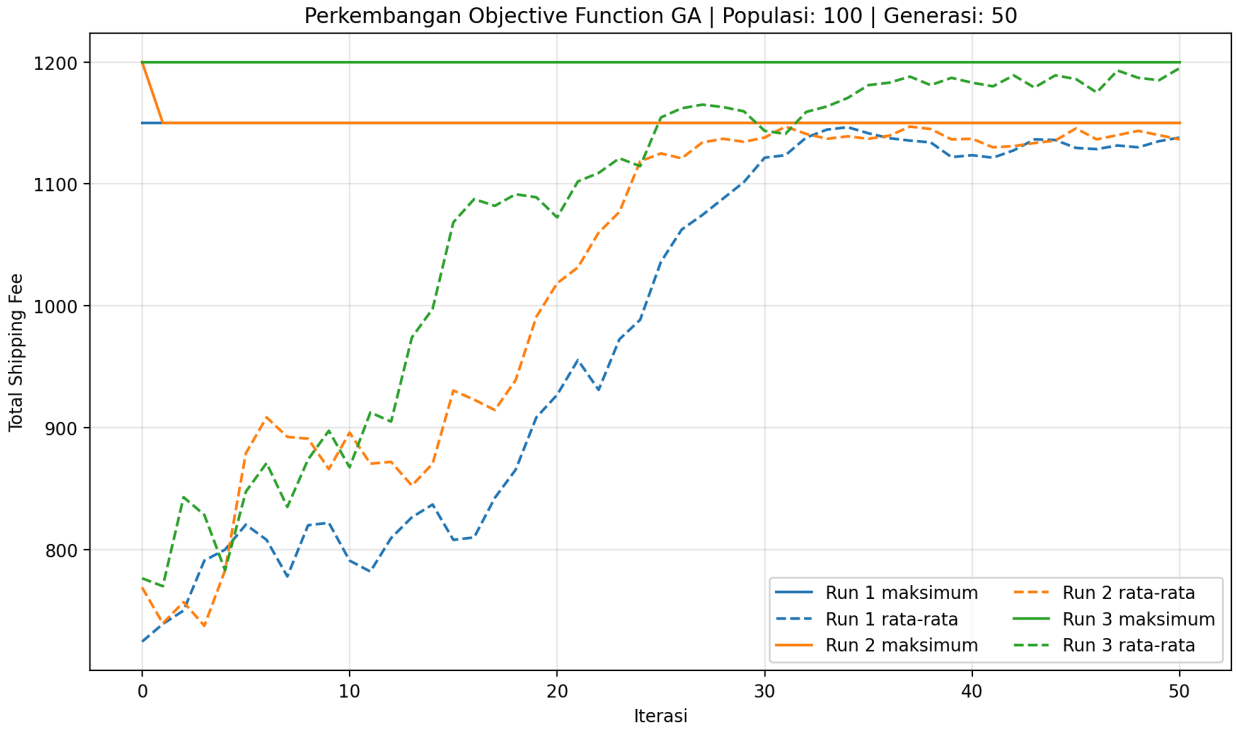


Tidak dimuat: V2, V3, V6, V7, V8, V9

Final State:



Function Objective Progression Histogram:

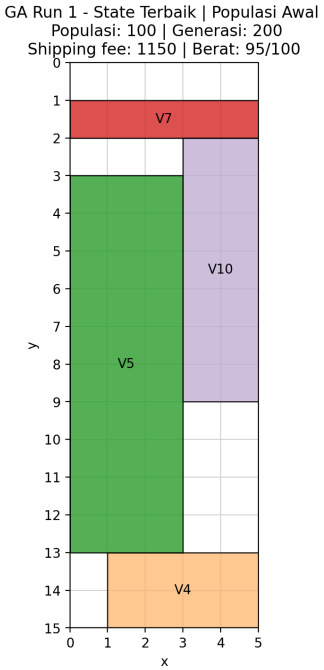


Eksperimen 3

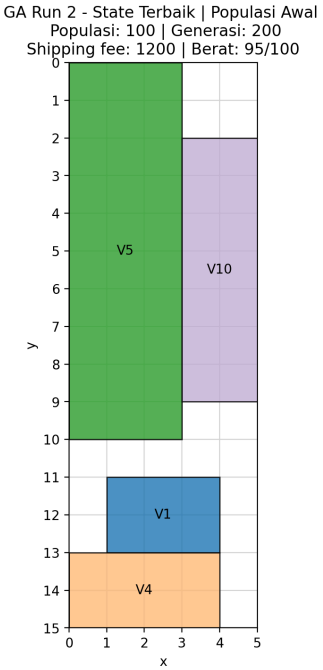
Data Eksperimen:

Run	Maks. Inisial	Skor Final	Maks. Pencarian	Durasi (detik)
1	1150	1050	1200	3.412
2	1200	1200	1200	0.913
3	1150	1150	1150	2.057

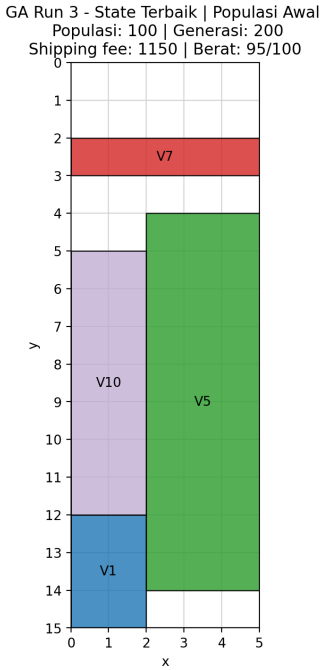
Initial State:



Tidak dimuat: V1, V2, V3, V6, V8, V9

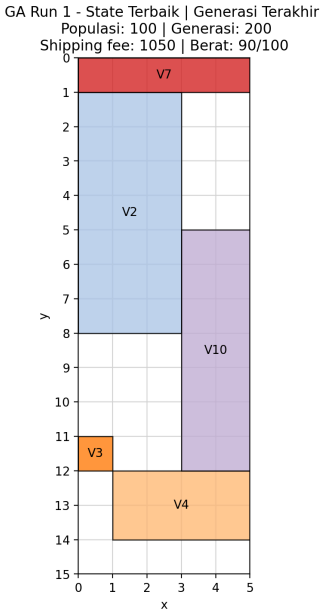


Tidak dimuat: V2, V3, V6, V7, V8, V9

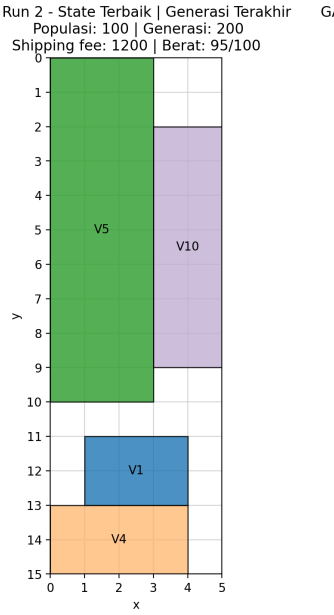


Tidak dimuat: V2, V3, V4, V6, V8, V9

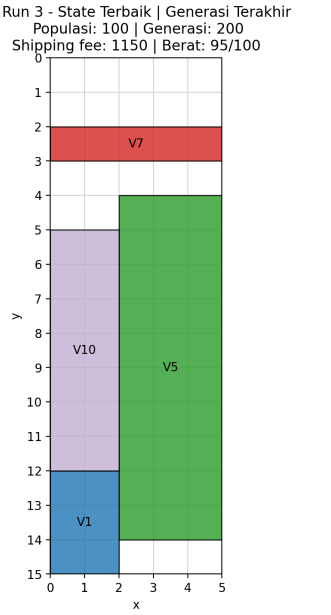
Final State:



Tidak dimuat: V1, V5, V6, V8, V9

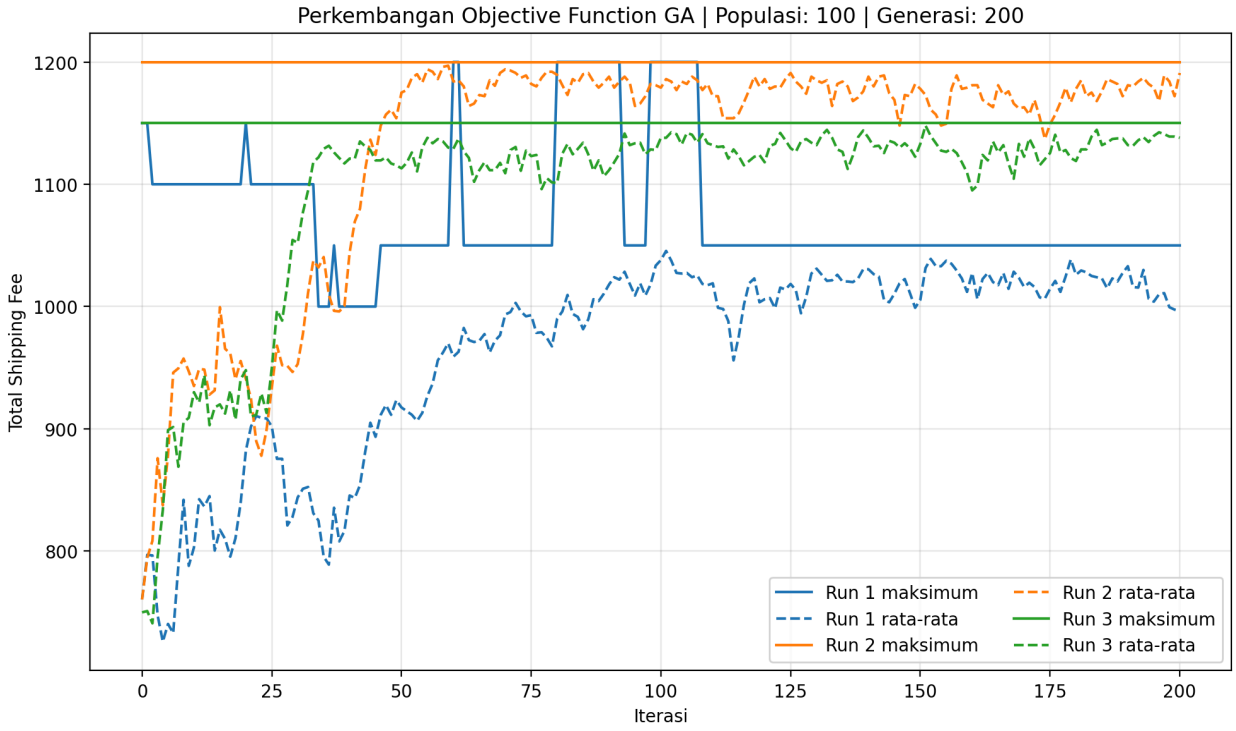


Tidak dimuat: V2, V3, V6, V7, V8, V9



Tidak dimuat: V2, V3, V4, V6, V8, V9

Function Objective Progression Histogram:

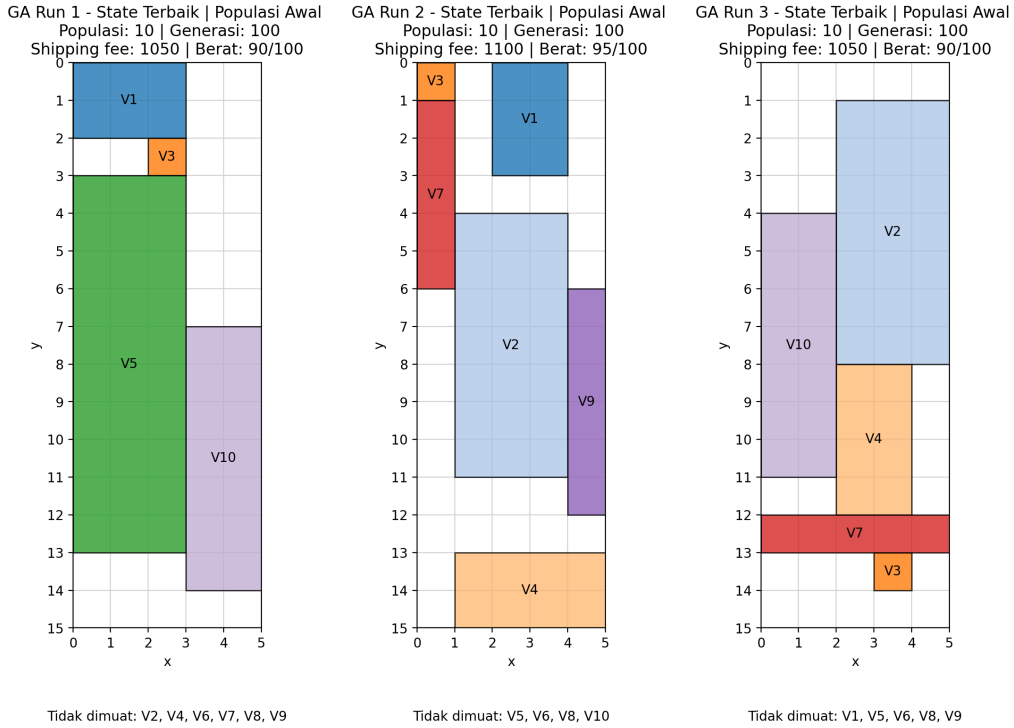


Eksperimen 4

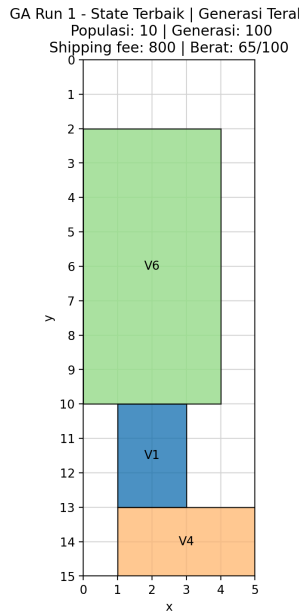
Data Eksperimen:

Run	Maks. Inisial	Skor Final	Maks. Pencarian	Durasi (detik)
1	1050	800	1050	0.140
2	1100	900	1100	0.044
3	1050	1000	1050	0.040

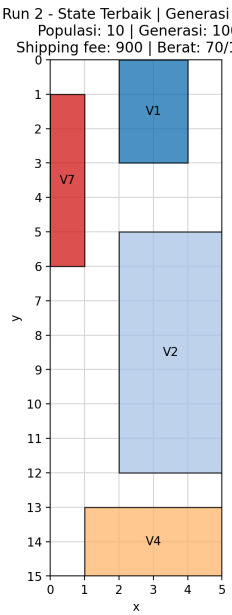
Initial State:



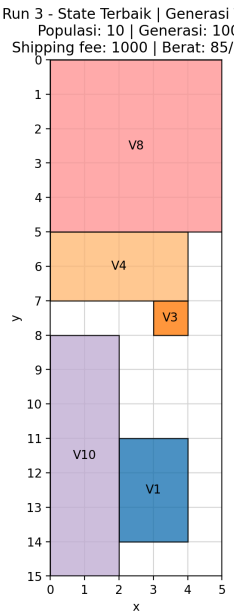
Final State:



Tidak dimuat: V2, V3, V5, V7, V8, V9, V10

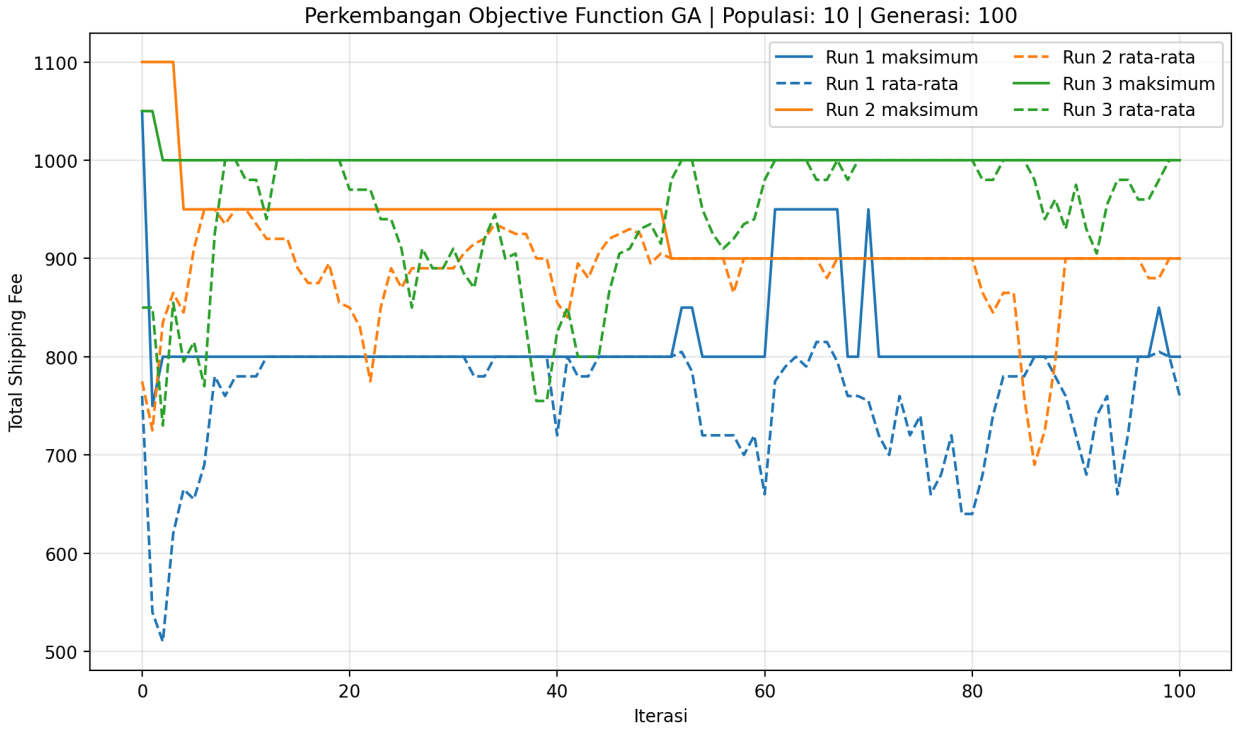


Tidak dimuat: V3, V5, V6, V8, V9, V10



Tidak dimuat: V2, V5, V6, V7, V9

Function Objective Progression Histogram:

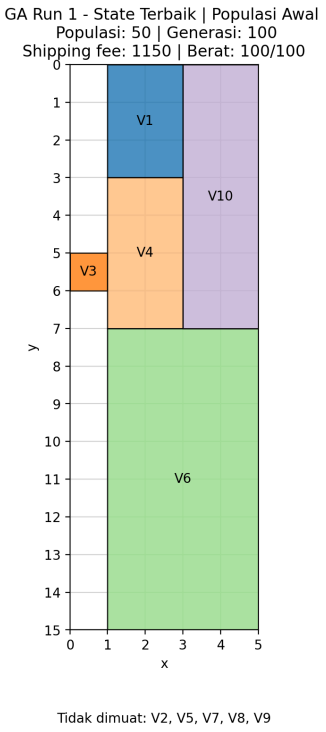


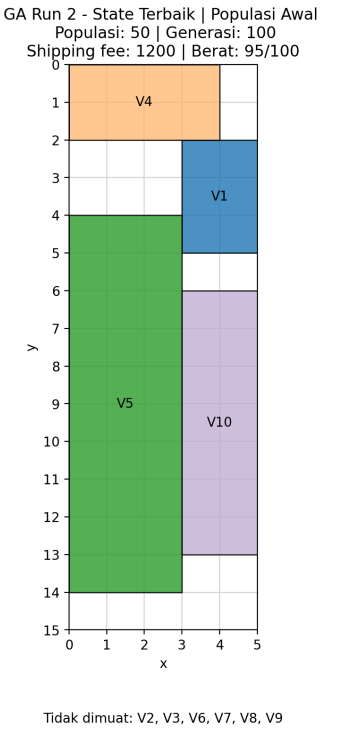
Eksperimen 5

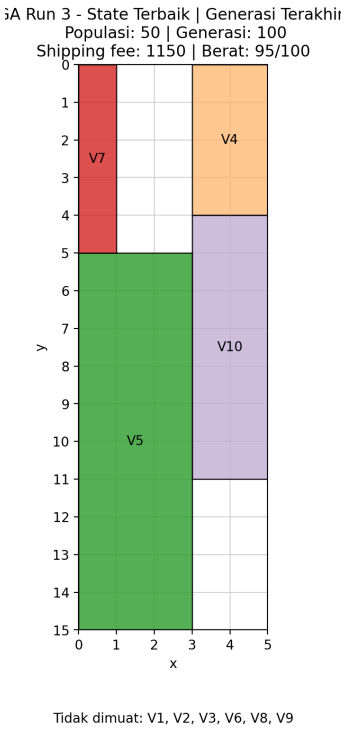
Data Eksperimen:

Run	Maks. Inisial	Skor Final	Maks. Pencarian	Durasi (detik)
1	1150	900	1150	0.248
2	1200	1100	1200	0.469
3	1150	1150	1200	0.656

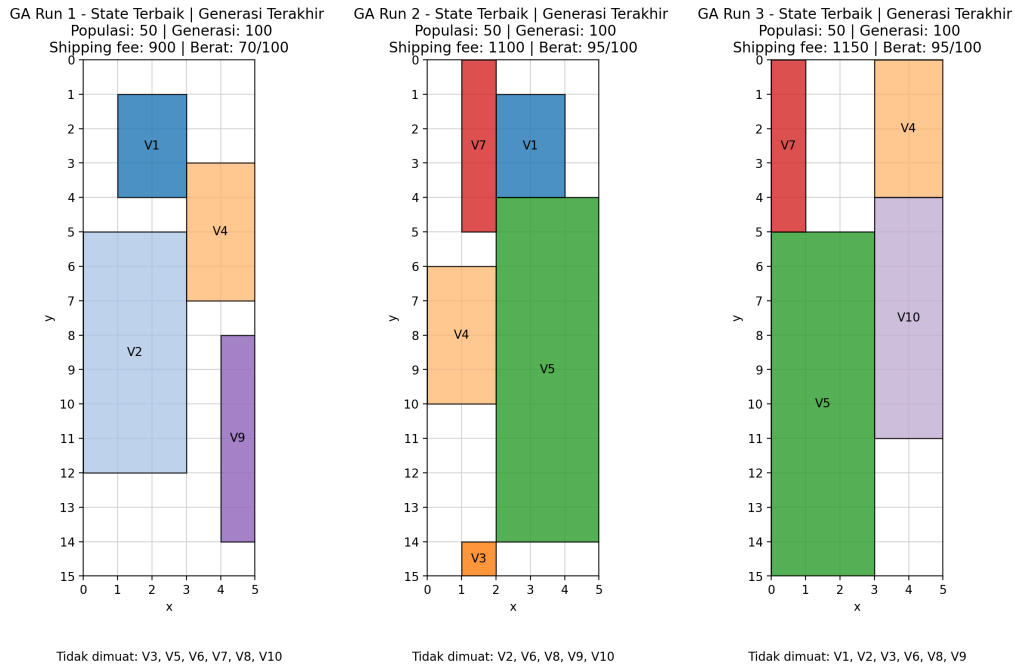
Initial State:



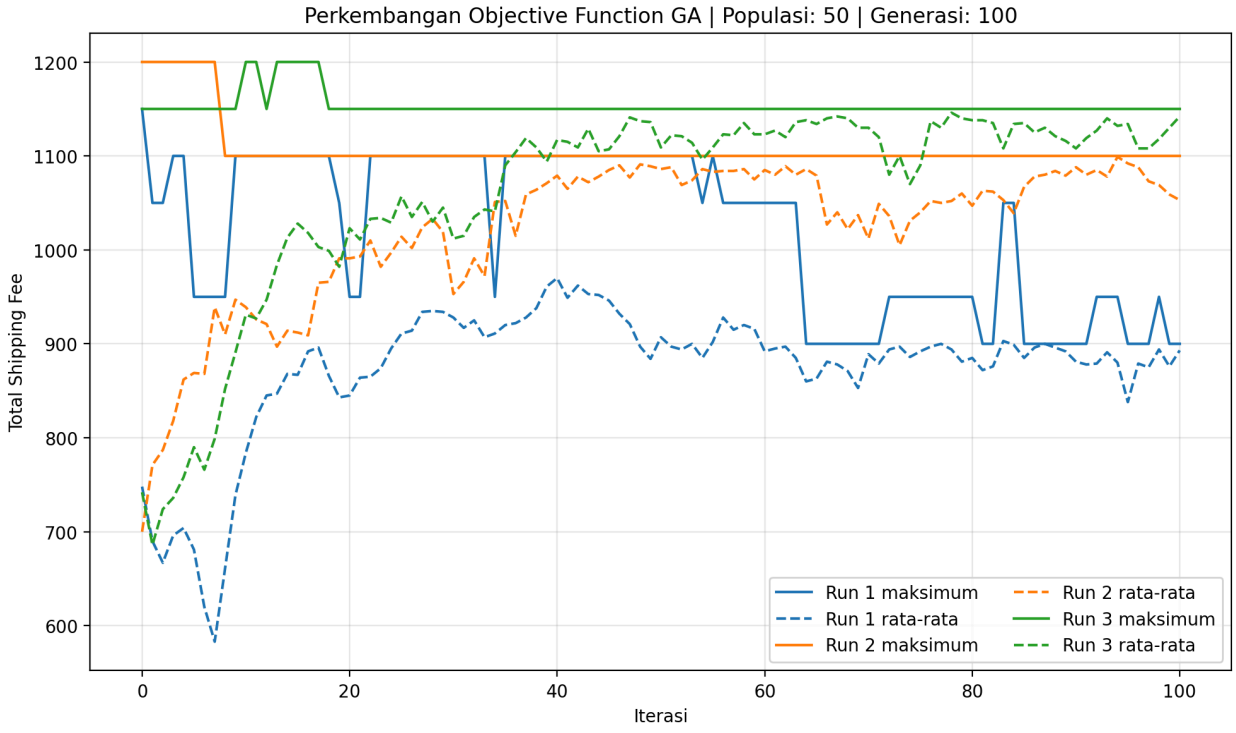




Final State:



Function Objective Progression Histogram:

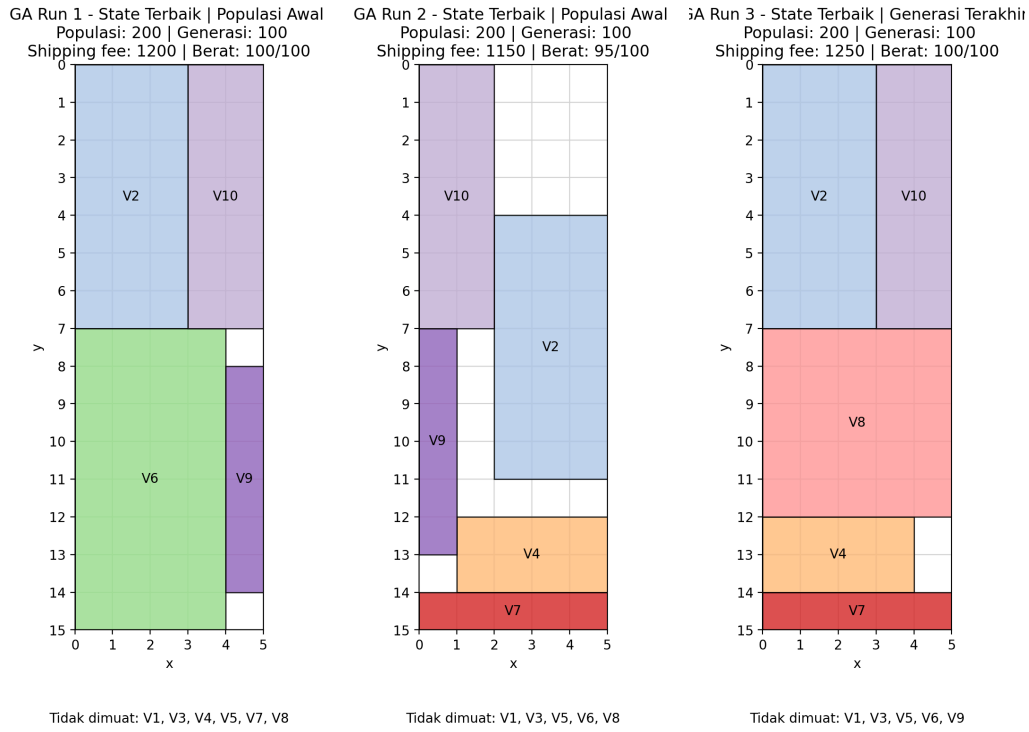


Eksperimen 6

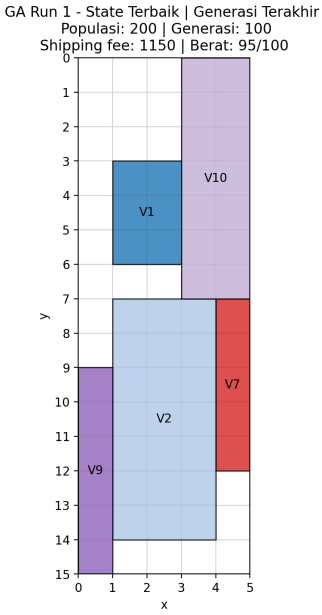
Data Eksperimen:

Run	Maks. Inisial	Skor Final	Maks. Pencarian	Durasi (detik)
1	1200	1150	1200	4.423
2	1150	1200	1200	4.235
3	1250	1250	1250	5.581

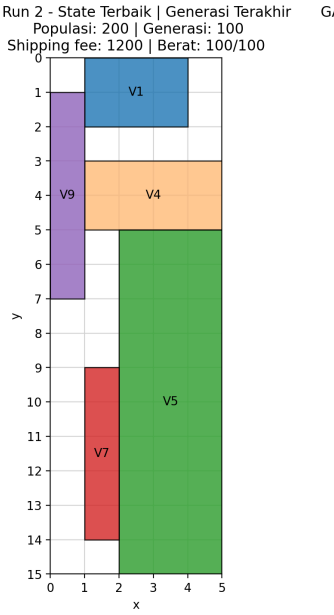
Initial State:



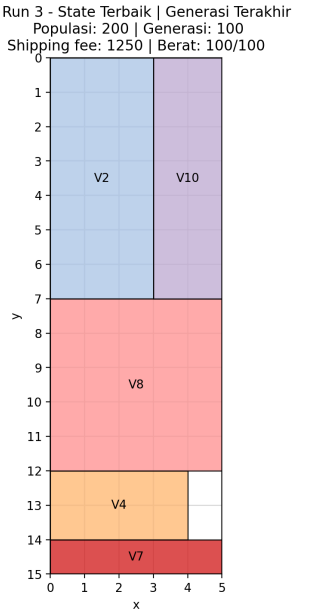
Final State:



Tidak dimuat: V3, V4, V5, V6, V8

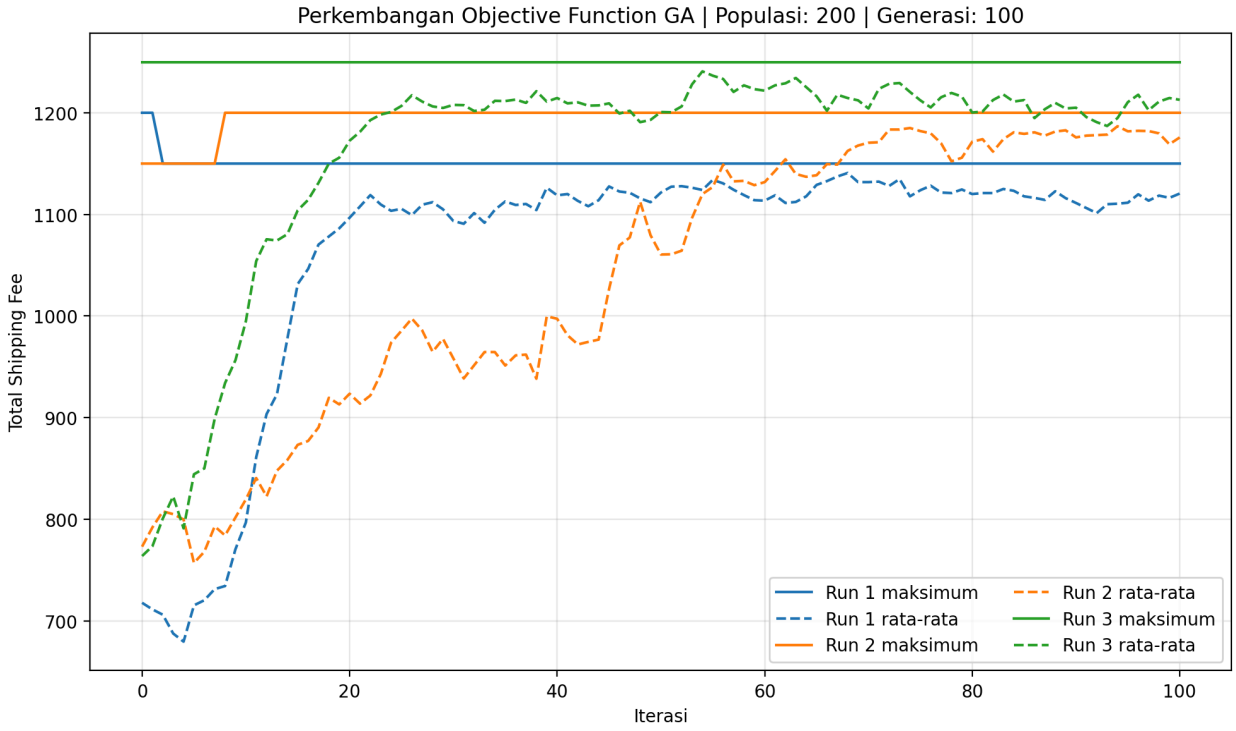


Tidak dimuat: V2, V3, V6, V8, V10



Tidak dimuat: V1, V3, V5, V6, V9

Function Objective Progression Histogram:



Hasil dan Analisis Eksperimen

Data Keseluruhan Eksperimen

No.	Populasi	Iterasi	Skor run 1	Skor run 2	Skor run 3	Mean skor akhir	Mean durasi (detik)
Control	100	100	1050	1200	1100	1116.67	0.903
1	100	10	1200	1050	950	1066.67	0.350
2	100	50	1150	1150	1200	1166.67	0.980
3	100	200	1050	1200	1150	1133.33	2.127
4	10	100	800	900	1000	900.00	0.075
5	50	100	900	1100	1150	1050.00	0.458
6	200	100	1150	1200	1250	1200.00	4.746

Pengaruh Jumlah Iterasi

Eksperimen 1 bila dibandingkan dengan kontrol, tidak ada perubahan signifikan selain durasi yang jauh lebih singkat dan konsistensi yang mengurang. Namun bila dilihat perbedaan dengan eksperimen 2, ada peningkatan konsistensi pencapaian nilai maksimum dengan 2 dari 3 run mendapatkan skor yang sama beserta rata-rata skor akhir yang meningkat. Namun bila dilihat dengan eksperimen 3 yang memiliki jumlah iterasi/generasi 2x lipat kontrol, tidak ada perubahan signifikan. Disini malah rata-rata skor akhir menurun dibanding eksperimen 2, dimana nilainya lebih rendah dari kontrol dengan durasi lebih dari 2x lebih lambat.

Pengaruh Jumlah Populasi

Eksperimen 4 bila dibandingkan dengan kontrol, memiliki mean skor yang lumayan lebih rendah (~19% penurunan) dengan maksimum dari ketiga run tidak ada yang melebihi 1000. Bila dibandingkan durasi rata-ratanya, eksperimen 4 lebih dari 12x kecepatan dari kontrol dan hampir 5x lebih cepat dari eksperimen 1. Eksperimen 5 memiliki skor-skor yang lebih tinggi dengan rata-rata 1050 dibandingkan 900 dari eksperimen 4. Namun memiliki durasi 6x lebih panjang. Eksperimen 6 memiliki skor akhir yang terbesar untuk keseluruhan eksperimen yaitu 1250 pada run 3 dengan rata-rata skor tertinggi yaitu 1200. Dari ke 6 eksperimen (dan kontrol), eksperimen ini lah yang memakan waktu tertinggi dengan 4.746 detik yaitu 13.5x lebih lambat dari kontrol dan 63x lebih lambat dari eksperimen 4.

Skor tertinggi yang teramati adalah 1250, ditemukan pada eksperimen 6 run 3 dan berhasil dipertahankan hingga generasi terakhir. Namun, skor tersebut sudah terdapat pada populasi awal, sehingga pencapaiannya tidak berasal dari peningkatan selama proses evolusi.

Pada 13 dari 21 run, skor akhir lebih rendah daripada skor maksimum yang pernah ditemukan selama pencarian sebesar 4,17% hingga 23,81%. Penurunan terkecil terjadi pada eksperimen 2 run 2, eksperimen 5 run 3, dan eksperimen 6 run 1, yaitu dari 1200 menjadi 1150. Penurunan terbesar terjadi pada eksperimen 4 run 1, yaitu dari 1050 menjadi 800. Hal ini menunjukkan keterbatasan implementasi dalam mempertahankan solusi terbaik karena seluruh populasi diganti setiap generasi kecuali anak hasil *crossover*, sementara solusi yang dikembalikan hanya individu terbaik dari generasi terakhir.

Meski demikian, mean nilai objective pada populasi akhir meningkat dibandingkan populasi awal pada 19 dari 21 run. Satu run menghasilkan rata-rata yang sama dan

satu run mengalami penurunan. Artinya, proses evolusi umumnya meningkatkan kualitas rata-rata populasi, tetapi belum selalu mempertahankan individu terbaiknya. Peningkatan rata-rata populasi tidak selalu berarti ditemukan solusi terbaik yang baru. Hanya 4 dari 21 run yang menemukan skor melebihi maksimum populasi awal, dan hanya 2 run yang mengakhiri pencarian dengan skor melebihi maksimum populasi awal. Dengan demikian, kualitas populasi awal masih berpengaruh besar terhadap hasil pencarian.

Berdasarkan eksperimen, parameter yang paling berpengaruh terhadap peningkatan efektifitas dan performa pencarian adalah jumlah populasi. Peningkatan jumlah populasi menunjukan peningkatan yang lebih konsisten terhadap pencarian dibandingkan jumlah iterasi/generasi. Namun jumlah populasi juga paling berpengaruh terhadap performa kecepatan algoritma dimana jumlah kecil sangat cepat dan jumlah besar membutuhkan waktu signifikan dibandingkan variansi lain.

Perbandingan Antar Algoritma

Perbandingan dilakukan berdasarkan nilai objektif akhir, konsistensi hasil, dan durasi pencarian pada data sepuluh kendaraan dengan geladak kapal berukuran 5×15 satuan grid serta kapasitas maksimum 100 satuan berat. Hill-Climbing dan Simulated Annealing masing-masing dijalankan menggunakan seed 1, 2, dan 3 dengan konfigurasi awal yang sama pada seed yang berpadanan. Genetic Algorithm menggunakan populasi awal yang dibentuk secara acak.

Tabel berikut merangkum tiga percobaan untuk setiap konfigurasi. Pada Genetic Algorithm, ditampilkan konfigurasi kontrol serta konfigurasi dengan rata-rata nilai akhir tertinggi dari seluruh konfigurasi yang diuji. Nilai objektif dinyatakan dalam juta rupiah,

sedangkan durasi merupakan waktu pencarian tanpa pembuatan visualisasi dan penyimpanan hasil.

Algoritma	Konfigurasi	Nilai akhir tiga percobaan	Rata-rata nilai akhir	Rata-rata durasi (detik)
Steepest Ascent Hill-Climbing	Seed 1, 2, dan 3	1050; 1050; 1100	1066,67	0,008204
Simulated Annealing	$T_0 = 800$; $\alpha = 0,998$; T_min = 1; maksimum 5000 iterasi	1150; 1150; 1050	1116,67	19,834477
Genetic Algorithm — kontrol	Populasi 100; 100 generasi; mutation rate 0,1	1050; 1200; 1100	1116,67	0,903400
Genetic Algorithm — eksperimen 6	Populasi 200; 100 generasi; mutation rate 0,1	1150; 1200; 1250	1200,00	4,746229

Simulated Annealing menghasilkan rata-rata nilai akhir 50 lebih tinggi daripada Hill-Climbing. Pada seed 1 dan 2, nilai akhir SA masing-masing lebih tinggi 100, sedangkan pada seed 3 nilainya lebih rendah 50 dibandingkan HC. Karena konfigurasi awalnya sama pada seed yang berpadanan, perbedaan ini memperlihatkan pengaruh proses pencarian terhadap hasil. Pada seed 3, SA sempat mencapai nilai 1150, tetapi mengembalikan state terakhir bernilai 1050. Hal ini menunjukkan bahwa kemampuan menemukan solusi yang lebih baik perlu disertai mekanisme untuk mempertahankannya.

Genetic Algorithm pada konfigurasi kontrol menghasilkan rata-rata nilai akhir yang sama dengan SA, yaitu 1116,67. Konfigurasi populasi 200 dan 100 generasi

memberikan rata-rata tertinggi sebesar 1200, dengan nilai akhir terbaik 1250. Namun, nilai 1250 tersebut sudah terdapat pada populasi awal percobaan ketiga. Dengan demikian, hasil GA juga dipengaruhi oleh kualitas populasi awal. Perbandingannya dengan HC dan SA perlu memperhatikan bahwa GA memulai pencarian dari banyak kandidat, sedangkan kedua algoritma lainnya menggunakan satu state awal.

Konsistensi hasil dapat dilihat dari selisih nilai akhir tertinggi dan terendah pada setiap konfigurasi. HC memiliki rentang 50, SA sebesar 100, GA kontrol sebesar 150, dan GA dengan populasi 200 sebesar 100. Di antara konfigurasi yang ditampilkan, HC menghasilkan penyebaran nilai akhir paling sempit. Sementara itu, GA dengan populasi 200 memberikan seluruh nilai akhir pada rentang 1150–1250, sehingga memperoleh rata-rata kualitas solusi lebih tinggi meskipun hasil antarpercobaannya tetap bervariasi.

Pada pengujian yang dicatat, HC memiliki rata-rata durasi paling singkat, yaitu 0,008204 detik, diikuti GA kontrol sebesar 0,903400 detik dan GA dengan populasi 200 sebesar 4,746229 detik. Singkatnya pencarian HC berkaitan dengan penghentian setelah satu atau dua pemeriksaan ketika tidak ditemukan peningkatan. Sementara itu, kedua konfigurasi GA menjalankan 100 generasi. Populasi yang lebih besar menambah jumlah individu yang diproses melalui seleksi, crossover, mutasi, dan pemeriksaan validitas pada setiap generasi.

SA mencatat rata-rata durasi terpanjang sebesar 19,834477 detik dengan 3339 iterasi pada setiap percobaan. Dalam implementasinya, fungsi random_neighbor() membangkitkan dan memvalidasi seluruh kandidat tetangga sebelum memilih satu secara acak. Proses tersebut diulang sepanjang pencarian, sehingga turut menambah waktu komputasi. Perbedaan ini menunjukkan bahwa jumlah iterasi tidak dapat dibandingkan secara langsung karena satu pemeriksaan HC, satu iterasi SA, dan satu

generasi GA memiliki beban kerja berbeda. Angka durasi tersebut menggambarkan waktu yang tercatat pada implementasi, konfigurasi, dan lingkungan pengujian masing-masing.

Kesimpulan dan Saran

Kesimpulan

Ketiga algoritma, Steepest Ascent Hill-Climbing, Simulated Annealing, dan Genetic Algorithm berhasil diterapkan untuk memilih serta menempatkan kendaraan pada geladak kapal dengan memenuhi batasan dimensi, larangan tumpang tindih, dan kapasitas angkut. Nilai terbaik yang mungkin pada data ini (optimum global) adalah 1250. Pada pengujiannya, rata-rata nilai objektif akhir HC adalah 1066,67, sedangkan SA dan GA kontrol sama-sama memiliki rata-rata 1116,67. GA dengan populasi 200 dan 100 generasi memiliki rata-rata tertinggi sebesar 1200, dan nilai akhir terbaik sebesar 1250.

HC mencatat waktu paling singkat, tetapi pencariannya terbatas karena hanya menerima perpindahan yang nilai objektifnya lebih baik. SA memungkinkan pencarian bahkan dengan konfigurasi nilai yang sama atau lebih rendah, sedangkan GA mengeksplorasi beberapa kandidat melalui populasi. Namun, kemampuan eksplorasi tersebut belum selalu mempertahankan solusi terbaik yang ditemukan. Kualitas populasi awal juga berperan pada GA, karena nilai 1250 telah terdapat pada populasi awal percobaan ketiga. Dengan demikian, hasil pengujian menunjukkan adanya pertimbangan antara kualitas solusi, pemeliharaan solusi terbaik, dan waktu komputasi.

Saran

1. Pengembangan HC dapat menambahkan sideways move dengan batas tertentu atau random restart untuk mengeksplorasi konfigurasi yang tidak dapat dijangkau melalui peningkatan nilai secara langsung.
2. SA sebaiknya dapat menyimpan solusi terbaik selama pencarian, terpisah dari solusi terakhir. Pada GA, penerapan elitism dapat mempertahankan individu terbaik saat membentuk generasi berikutnya, sehingga solusi berkualitas tidak mudah hilang.
3. Pembangkitan tetangga SA dapat dioptimalkan agar memperoleh kandidat valid tanpa membentuk seluruh daftar tetangga pada setiap iterasi. Evaluasi lanjutan dapat menggunakan variasi data kendaraan dan kondisi perangkat yang sama untuk menilai pengaruh perubahan implementasi terhadap kualitas solusi serta durasi pencarian.
4. Eksperimen lanjutan sebaiknya memakai lebih banyak seed atau pengulangan pada data dan perangkat yang sama, agar perbedaan kualitas solusi dan durasi antar algoritma dapat diuji lebih meyakinkan.

Pembagian Tugas

Nama	NIM	Tugas
Ghaisan Nufail	18224090	Simulated Annealing
Andreas Saputra Tambun	18224110	Hill Climbing
Daniel Wicaksana Gojdali	18224114	Genetic Algorithm

Referensi

[1] Institut Teknologi Bandung, “Beyond Classical Search,” salindia kuliah IF3070 Dasar Inteligensi Artifisial, Semester I Tahun Akademik 2026/2027 [Online]. Available: Edunex. [Accessed: 17 October 2026].

[2] University of California, Berkeley, “1.5 Local Search,” Introduction to Artificial Intelligence, CS 188. [Online]. Available: https://inst.eecs.berkeley.edu/~cs188/textbook/search/local.html [Accessed: 18 October 2026].

[3] Dipen Desai, “Steepest-Ascent Hill Climbing Algorithm,” YouTube, 16 November 2021. [Online]. Available: Steepest-Ascent Hill Climbing Algorithm [Accessed: 18 October 2026].

[4] DigitalSreeni, “314 - How to code the genetic algorithm in python?,” YouTube, 21 February 2024. [Online]. Available: 314 - How to code the genetic algorithm in python? [Accessed: 18 October 2026].

[5] Tyler Brand , “Traveling Salesman Problem - Simulated Annealing - Python Animation,” YouTube, 12 Jan 2025 . [Online]. Available: Traveling Salesman Problem - Simulated Annealing - Python Animation [Accessed: 20 October 2026].

Lampiran

LAMPIRAN 1

DEKLARASI PENGGUNAAN AI

LAMPIRAN 2

LINK REPOSITORY GITHUB